

数据结构

Ch9 查找

北京邮电大学 计算机学院

第9章 查找

- 9.0 概念和术语
- 9.1 静态查找表
 - 9.1.1 顺序表查找 9.1.2 有序表查找 (补充内容: 跳表)
 - 9.1.3 静态树表查找 9.1.4 索引顺序表查找
- 9.2 搜索树
 - 9.2.1 二叉排序树
 - 9.2.2 平衡二叉树 (补充内容: 伸展树、红黑树)
 - 9.2.3 B-树 (平衡的多路查找树)
 - 9.2.4 B+树
 - 9.2.5 键树
- 9.3 哈希表

9.0 问题提出

在我们的日常生活中，离不开形形色色的查询操作，如英文词典、学生成绩单、工资表、电话簿、图书检索。可以说，信息检索是计算机最重要的一种应用。这里，我们所说的**查询**、**检索**和**查找**是同一个概念。

查找是指在一组**数据集合**找出满足特定条件的**特定元素**的位置。

特定元素：关键字值等于给定值



The screenshot shows the SF Express website interface. At the top, there is a search form with two input fields: '公司名称' (Company Name) and '快递单号' (Express Number). The '公司名称' field has a dropdown menu with '顺丰速运' (SF Express) selected. The '快递单号' field has a placeholder text '请输入您的快递单号' (Please enter your express number). To the right of the '快递单号' field is a blue button labeled '查询' (Search). Below the search form, there is a section for SF Express information, including the SF logo, the company name '顺丰速运', the website address '官网地址: www.sf-express.com', and the customer service phone number '客服电话: 95338'. At the bottom of this section are two buttons: '网点查询' (Branch Search) and '网上寄件' (Online Shipping). In the bottom right corner of the page, there is a logo for '快递100' (Kaidi 100) and the text '数据来自快递100' (Data from Kaidi 100).



9.0 相关概念和术语

快递单号	寄件人姓名	寄件人手机号	寄件人地址	收件人姓名	收件人手机号	收件人地址	揽件时间	揽件员
114750081239	张三	13812345678	北京市海淀区	李四	13912345678	上海市黄浦区	2000/1/1	小明
199130201363								
023221407143								
205310838056								
574748062096								

查找表 是一种以**集合**为逻辑结构，以**查找**为核心运算的数据结构。

查找 指定某个值，在查找表中确定是否存在一个记录，该记录的**关键字**等于给定值。

关键字 记录(数据元素)中的某个**数据项**的值。

主关键字 该关键字可以唯一地标识一个记录。如学生花名册中的学号

次关键字 该关键字不能唯一标识一个记录。如学生花名册中的姓名

9.0 相关概念和术语

查找表**以集合为逻辑结构**，因此数据元素之间是没有“关系”的，在实现时就不受“关系”的约束，根据实际应用去组织查找表，以提高查找的效率，常见的存储结构有：**顺序结构、链式结构、索引结构、散列结构**。

基于在查找表中操作的不同，查找表又分为以下两种：

- **静态查找表** 对查找表的查找仅是以查询为目的，**不改动查找表中的数据**。
- **动态查找表** 查找过程中还会有**插入**不存在的数据元素或**删除**存在的数
据元素操作。

查找成功 查找表中**存在**满足查找条件的记录。

查找不成功 查找表中**不存在**满足查找条件的记录。

内查找 整个查找过程都在**内存中**进行。

外查找 在查找过程中需要**访问外存**。

平均查找长度ASL (**A**verage **S**earch **L**ength)

- 为确定记录在查找表中的位置, 需将关键字和给定值**比较次数的期望值**。
- 查找方法时效的度量

查找成功时的ASL计算方法:

$$ASL = \sum_{i=1}^n p_i c_i \quad \sum_{i=1}^n p_i = 1$$

n: 记录的个数

p_i : 查找第*i*个记录的概率,

(不特别声明时认为等概率 $p_i = 1/n$)

c_i : 找到第*i*个记录所需的比较次数

[查找表上的基本操作]

- 1) 查找表初始化: `InitTable(&T)`
- 2) 销毁查找表: `DestroyTable(&T)`
- 3) **检索**某个特定的数据元素: `SearchTable(T, key)`
- 4) **插入**一个数据元素: `InsertTable(&T, e);`
- 5) **删去**某个数据元素: `DeleteTable(&T, key);`
- 6) 遍历: `TraverseTable(T, Visit());`

9.1 静态查找表

9.1.1 顺序表查找

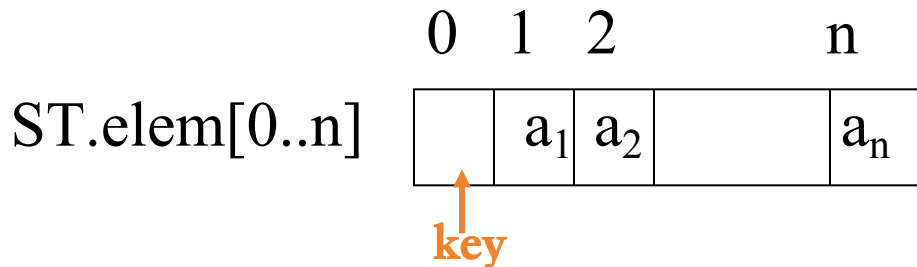
9.1.2 有序表查找

9.1.3 静态树表查找

9.1.4 索引顺序表查找

9.1.1 顺序表的查找

静态查找表以顺序表表示



```
typedef struct {  
    ElemType *elem;  
    int length;  
}SSTable;
```

[算法描述]

```
int Search_Seq1( SSTable ST, KeyType key)
```

```
{
```

```
    int Search_Seq2( SSTable ST, KeyType key)
```

```
    {  
        ST.elem[0].key=key;    //放置“哨兵”
```

```
        for (i=ST.length; ST.elem[i].key!=key; i--);
```

```
        return i;
```

```
    }//Search_Seq2
```

```
}//Search_Seq1
```

[性能分析]

空间：一个辅助空间。

时间：

1) 查找成功时的平均查找长度

设表中各记录查找概率相等

$$ASL_s(n) = \sum_{i=1}^n P_i C_i = (1+2+\dots+n)/n = (n+1)/2$$

2) 查找不成功时的平均查找长度 $ASL_f = n+1$

[算法特点]

- 算法简单，对表中元素排列顺序无任何要求
- n 很大时查找效率较低，一种改进措施：非等概率查找时，可将查找概率高的记录尽量排在表前部。
- 逐个查找的方法也可用于线性链表表示的静态查找表

[程序设计技巧] 设置**监视哨**，查找时**不必**每一步**检测是否越界**，提高效率。

思考：若采用线性链表存储静态查找表，为避免每一步的越界检测，用什么形式的链表为好：单链表、单循环链表、双循环链表？

9.1.2 有序表的查找

<https://www.cs.usfca.edu/~galles/visualization/Search.html>

有序表： 满足 $r[i].key \leq r[i+1].key$, $1 \leq i < n$

*有序表仍可用顺序查找，但在找不到时不需比较到表尾，只需比较到比给定值大的记录就可终止。

[折半(对半/二分)查找法]

0	1	2	3	4	5	6	7	8	9	10	11
	05	13	19	21	37	56	64	75	80	88	92
↑low			↑mid				↑high				
$= \lfloor (low+high)/2 \rfloor$											

查找 **K=21**

l	h	m
1	11	6
1	5	3
4	5	4

查找 **K=85**

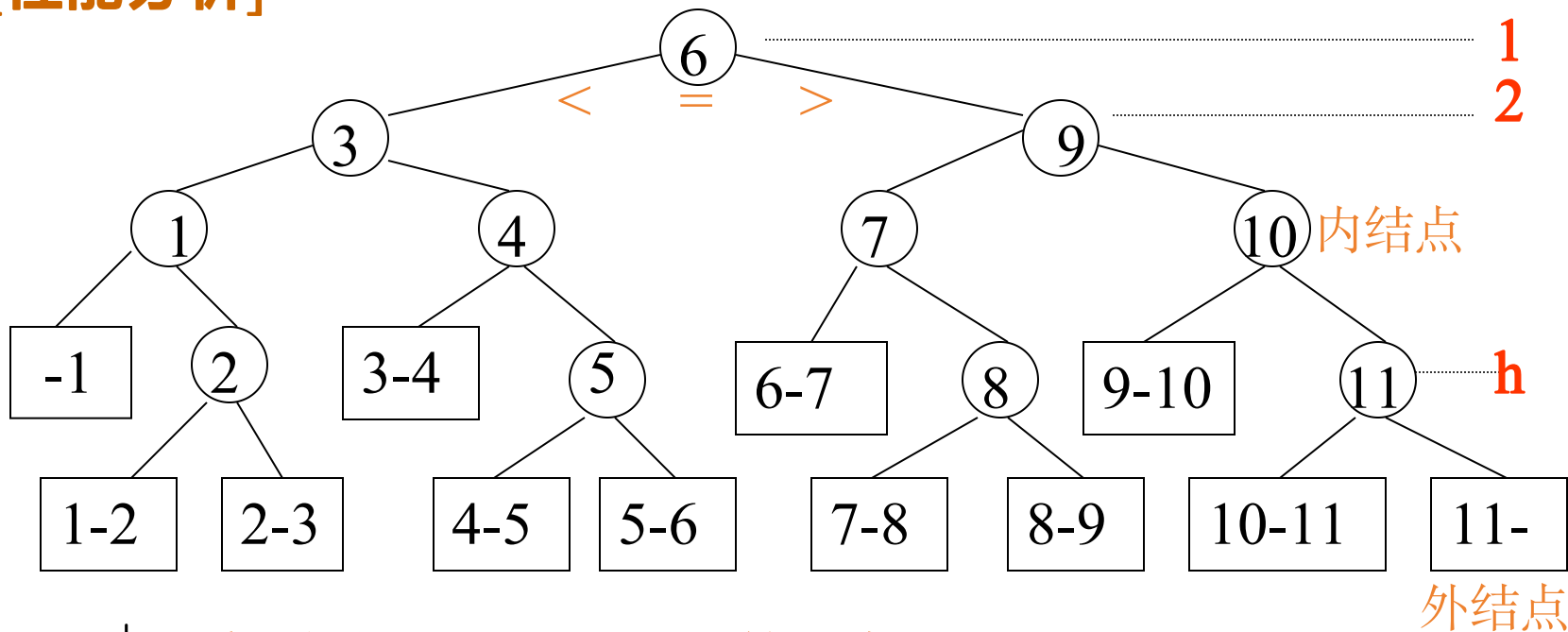
l	h	m
1	11	6
7	11	9
10	11	10
10	9	

[算法描述]

```
int Search_Bin ( SSTable ST, keytype key)
{   low= 1;      high = ST.length;      // 置区间初值
    while ( low <= high )  { // 要点1: 结束条件
        mid = ( low+high )/2; // 要点2: mid计算
        if (key==ST.elem[mid].key)
            return mid;      // 找到待查元素
        else
            if ( key< ST.elem[mid].key) // 要点3: low和high更新
                high=mid-1; // 继续在前半区间进行查找
            else low=mid+1; // 继续在后半区间进行查找
    }
    return 0;      // 顺序表中不存在待查元素
} //Search_Bin
```

[性能分析]

判定树 (描述查找过程的二叉树)



$h = \lfloor \log_2 n \rfloor + 1$ {同完全二叉树, 二叉树性质4}

成功查找时的平均查找长度:

($n > 50$)

$$ASL_s = \frac{1}{n} \sum_{j=1}^h j \cdot 2^{j-1} = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

例: $ASL_s = (1 \cdot 1 + 2 \cdot 2 + 3 \cdot 4 + 4 \cdot 4) / 11 = 3$

不成功查找时的查找长度: $h-1$ 或 h 次, (因为外结点是不存在的不增加比较次数)

- 二分查找是非常高效的
 - 例：在 $n=2^{32}$ ，约42亿数据中进行二分查找，最多只需要比较32次。
 - $O(\log n)$ 有的时候比时间复杂度是 $O(1)$ 的算法还要高效。
- 二分查找应用场景的局限性
 - 顺序存储，普通链表不可以
 - 维护有序的成本不能太高：不能有频繁的插入、删除，最好是一次排序，大量的查找操作，从而分摊排序的成本。
 - 数据量太小不适合用二分查找。

- 二分查找变体问题
 - 查找第一个值等于给定值的元素

```
int Search_Bin ( SSTable ST, keytype key)
{  low= 1;    high = ST.length;
   while ( low <= high )
       mid = ( low+high )/2;
       if (key==ST.elem[mid].key) { //相等时还需要进一步确认
           if((mid == 1 || ST.elem[mid-1].key !=key))
               return mid;
           else high=mid-1
       }
       else if ( key< ST.elem[mid].key) //要找的key比当前mid元素小
           high=mid-1;
       else low=mid+1;
   }
   return 0; // 顺序表中不存在待查元素
} //Search_Bin
```


- 二分查找变体问题

- 查找最后一个值等于给定值的元素

- 略

- 查找第一个值大于或等于给定值的元素

```
if(key <= ST.elem[mid].key){
```

```
    if((mid == 1 || key > ST.elem[mid-1].key)) return mid;
```

```
    else high = mid - 1
```

```
}else{
```

```
    low = mid + 1;
```

```
}
```

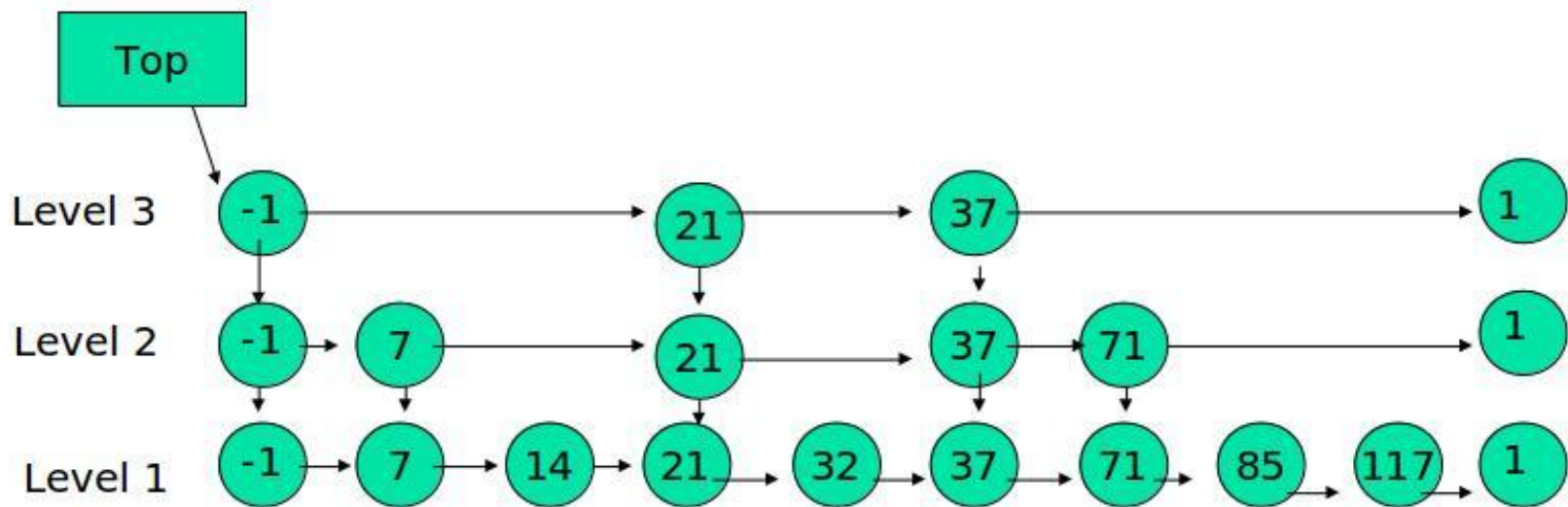
- 查找最后一个值小于或等于给定值的元素

- 略

跳表 (skiplist)

- 实现简单，性能优良，插入、删除、查找的复杂度均为 $O(\log N)$ 。

Redis的有序集合 (Sorted Set) 用跳表实现。

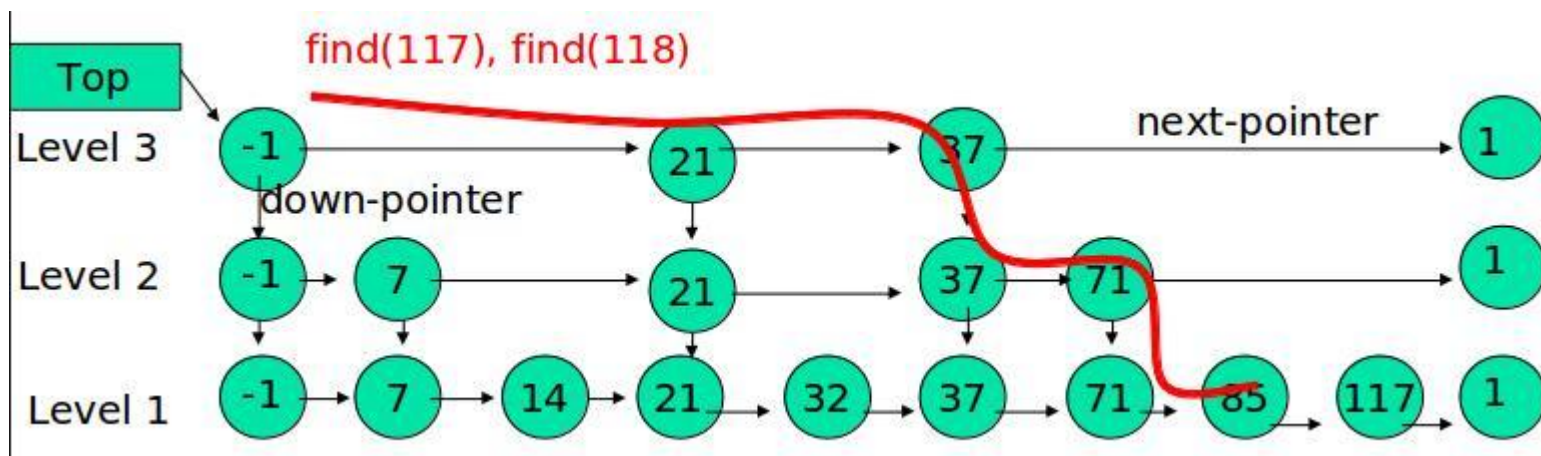


- 核心思路：对链表建立多层“索引”，上图中-1表示 `INT_MIN`，链表的最小值，1表示 `INT_MAX`。如果索引层足够，相对于实现了二分查找。

跳表 (skiplist)

- 实现简单，性能优良，插入、删除、查找的复杂度均为 $O(\log N)$ 。

Redis的有序集合 (Sorted Set) 用跳表实现。

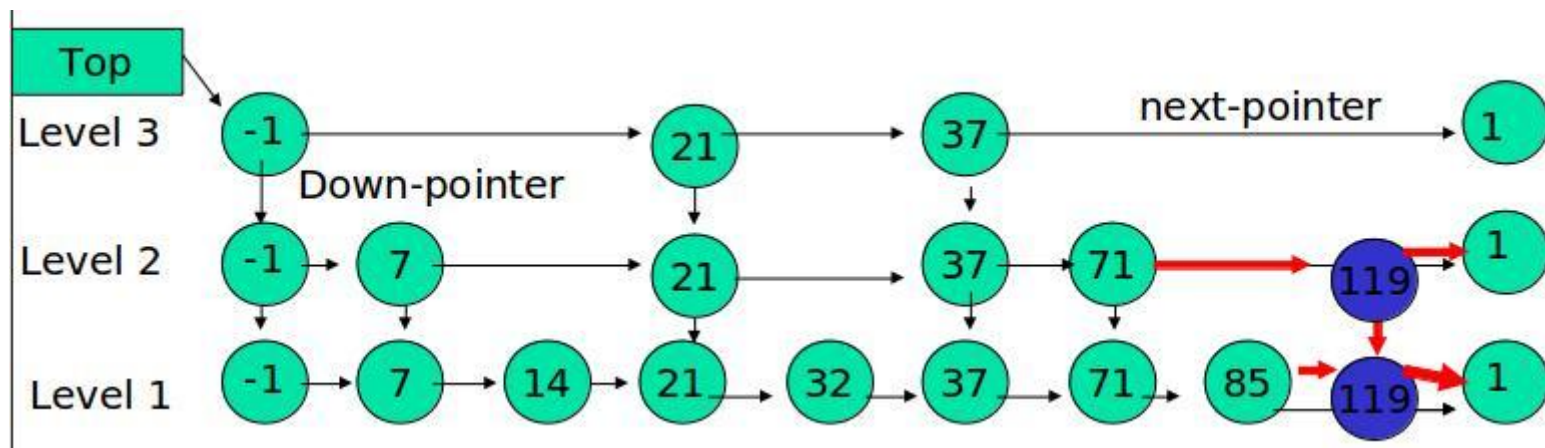


- 最上层(Level3): 比37大, 但比链表最大值小, 从37进入下一层查找;
- 第二层(Level2): 比71大, 但比链表最大值小, 从71进入下一层查找;
- 最下层(Level1): 比85大, 继续比较117, 等于117, 找到了节点。

跳表 (skiplist)

- 实现简单，性能优良，插入、删除、查找的复杂度均为 $O(\log N)$ 。

Redis的有序集合 (Sorted Set) 用跳表实现。

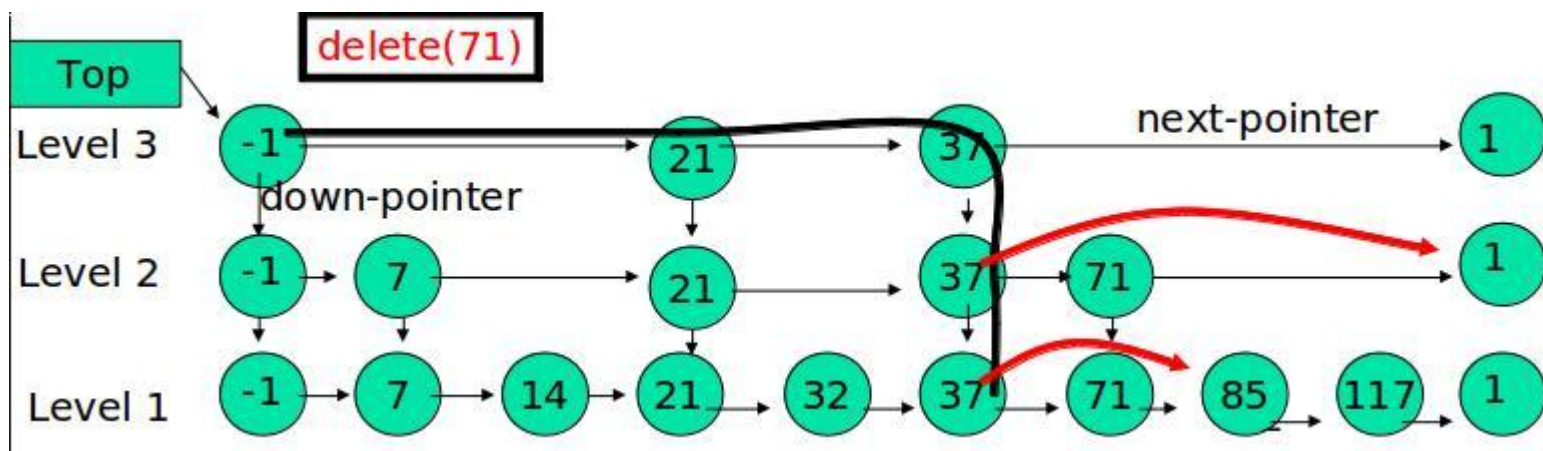


插入: 随机选择一个k，然后从第k索引层开始往下每层都插入新结点，例如上例中随机到的层数是2，然后从level2 level1都会插入新结点119

跳表 (skiplist)

- 实现简单，性能优良，插入、删除、查找的复杂度均为 $O(\log N)$ 。

Redis的有序集合 (Sorted Set) 用跳表实现。



删除： 在各个层中找到包含要删除的节点，然后删除该节点

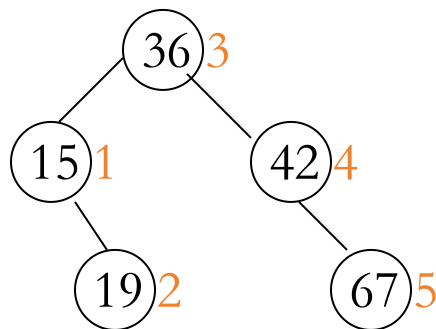
9.1.3 静态树表的查找

[问题出发点]

讨论有序表中各记录查找概率不等时，查找成功时的ASL。

	1	2	3	4	5
关键字	15	19	36	42	67
查找概率	0.1	0.2	0.1	0.4	0.2

折半查找

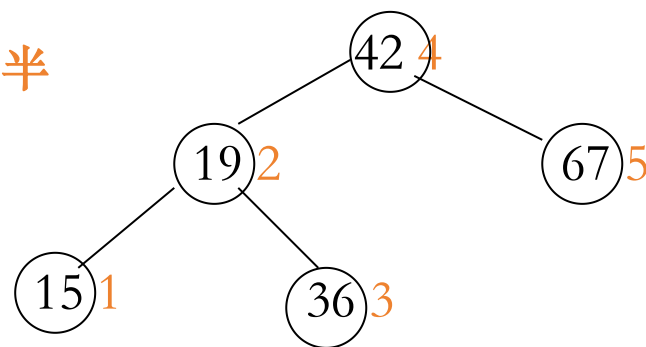


$$ASL = \sum_{i=1}^n P_i C_i = 2.3$$

$$= 0.1 \times 1 + 0.1 \times 2 + 0.4 \times 2 + 0.2 \times 3 + 0.2 \times 3$$

高概率为根

+折半



$$ASL = \sum_{i=1}^n P_i C_i = 1.8$$

$$= 0.4 \times 1 + 0.2 \times 2 + 0.2 \times 2 + 0.1 \times 3 + 0.1 \times 3 = 1.8$$

[静态最优查找树]

定义：查找性能最佳的判定树。

性质：带权内路径长度之和PH为最小值。

$$PH = \sum_{i=1}^n w_i h_i \quad \text{与ASL成正比}$$

其中n：二叉树上结点的个数(有序表长度)

h_i ：第i个结点在二叉树上的层次数

$w_i = c p_i$ ：c为某个常量；

p_i ：第i个结点的查找概率

最优查找体现的原则：

- 1) 最先访问的结点应是访问概率最大的结点；
- 2) 每次访问应使结点两边尚未访问的结点的被访概率之和尽可能相等。

[次优查找树的构造方法]

- PH值近似为最小
- 比静态最优查找树易于构造，时间开销少

查找性能只差1%-2%，很少差3%以上

已知按关键字有(升)序的记录序列：

$(r_1, r_{l+1}, \dots, r_{i-1}, \mathbf{r_i}, r_{i+1}, \dots, r_{h-1}, r_h)$

1)选取记录 $\mathbf{r_i}$ 作为根结点：

计算所有 Δp_i ($1 \leq i \leq h$)

找出最小 Δp_i 的对应的 $\mathbf{i}$ ；

$$\Delta p_i = \left| \sum_{j=i+1}^h w_j - \sum_{j=l}^{i-1} w_j \right|$$

$\mathbf{i}$ 后面结点的权值和

$\mathbf{i}$ 前面结点的权值和

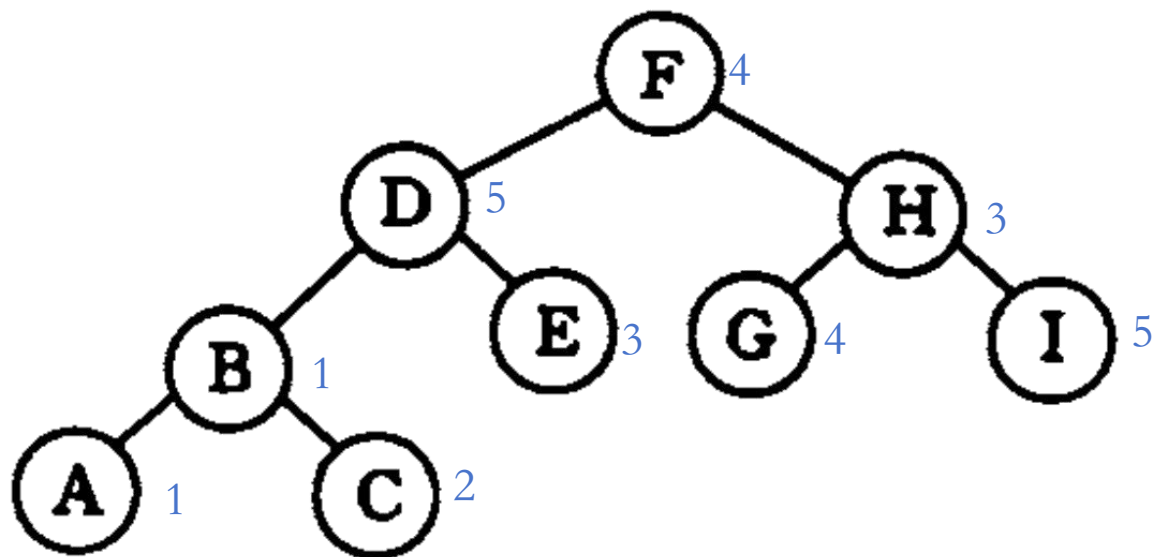
前后权值和尽可能相等

由于在构造次优查找树的过程中，没考虑单个关键字的权限，若 $\mathbf{i}$ 相邻关键字的权值大，则确定为权值大的关键字对应的 $\mathbf{i}$



[例]

j	1	2	3	4	5	6	7	8	9	
关键字	A	B	C	D	E	F	G	H	I	有序
权 值	1	1	2	5	3	4	4	3	5	
SW_j	1	2	4	9	12	16	20	23	28	
ΔP_j	[27	25	22	15	7]	0	[8	15	23]	
ΔP_j	11	9	6	1	9		8	1	7	
...	...	...	...	...	...	...	...	...	...	



已知按关键字有(升)序的记录序列:

($r_1, r_{1+1}, \dots, r_{i-1}, r_i, r_{i+1}, \dots, r_{h-1}, r_h$)

$$\Delta p_i = \left| \sum_{j=i+1}^h w_j - \sum_{j=l}^{i-1} w_j \right|$$

简化计算 Δp_i 的方法:

设定累计权值和 $sw_i = \sum_{j=1}^i w_j$

则

← 第i+1项到第h项的累计和=前h项的累计和-前i项的累计和

$$\sum_{j=i+1}^h w_j = sw_h - sw_i$$

$$\sum_{j=l}^{i-1} w_j = sw_{i-1} - sw_{l-1}$$

$$\begin{aligned} \Delta p_i &= |(sw_h - sw_i) - (sw_{i-1} - sw_{l-1})| \\ &= |(sw_h + sw_{l-1}) - sw_i - sw_{i-1}| \end{aligned}$$

设 $w_0=0, sw_0=0$

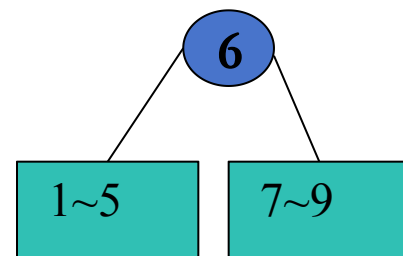
2) r_i 的左子树: ($r_1, r_{1+1}, \dots, r_{i-1}$)构造的次优查找树

3) r_i 的右子树: ($r_{i+1}, \dots, r_{h-1}, r_h$)构造的次优查找树

设 $w_0=0$, $sw_0=0$

$$\Delta p_i = |(sw_h + sw_{l-1}) - sw_i - sw_{i-1}|$$

如果 l 从 1 开始, 可以简化为: $\Delta p_i = |sw_h - sw_i - sw_{i-1}|$



[例]

j	1	2	3	4	5	6	7	8	9	
关键字	A	B	C	D	E	F	G	H	I	有序
权 值	1	1	2	5	3	4	4	3	5	
SW_j	1	2	4	9	12	16	20	23	28	
ΔP_j	[27	25	22	15	7]	0	[8	15	23]	
ΔP_j	11	9	6	1	9		8	1	7	
...	...	...	...	...	...	...	...	...	...	

$|28-1-0|$
 $|12-1-0|$

$|28-2-1|$
 $|12-2-1|$

$|28-12-9|$

$|28-20-16|$
 $|28+16-20-16|$

$|28+16-28-23|$

[次优查找树上的查找步骤]

设给定的查找值为K

将根结点作为当前考察的结点

1) 若当前结点指针为NULL，则返回空指针

2) 将当前结点的关键字key与K比较，

若 $K = \text{key}$ ，则返回当前结点的指针；

若 $K > \text{key}$ ，将其右孩子作为当前结点，转1)；

若 $K < \text{key}$ ，将其左孩子作为当前结点，转1)；

[查找性能]

比较次数不超过树的深度， $O(\log_2 n)$

9.1.4 索引顺序表的查找

[表的构造]

索引表 $\text{index}[1..b]$

最大关键字值

起始地址

主表 $r[1..n]$ (分块有序)

key

其它项

22	42	86
1	5	9

12	22	13	8	28	33	38	42	86	76	50	63
1	2	3	4	5	6	7	8	9	10	11	12

```
typedef struct {
    KeyType maxkey;
    int    stadr;
}indexItem; //索引项
typedef struct {
    indexItem *elem;
    int    blocknum;
}indexTable; //索引表
```

[分块查找步骤]

- 1) 折半或者顺序查找索引表, 确定所在块
- 2) 在已确定的块中顺序查找

[性能分析]

索引顺序查找的平均查找长度 =
查找“索引”的平均查找长度
+ 查找“块”的平均查找长度

设b为索引表长度，s为块中记录个数

顺序查找索引表+顺序查找被确定的块

$$ASL_{bs} = \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{1}{2}(b + s) + 1 = \frac{1}{2} \left(\frac{n}{s} + s \right) + 1$$

当s取 $\sqrt{n}$ 时， ASL_{bs} 取最小值 $\sqrt{n} + 1$

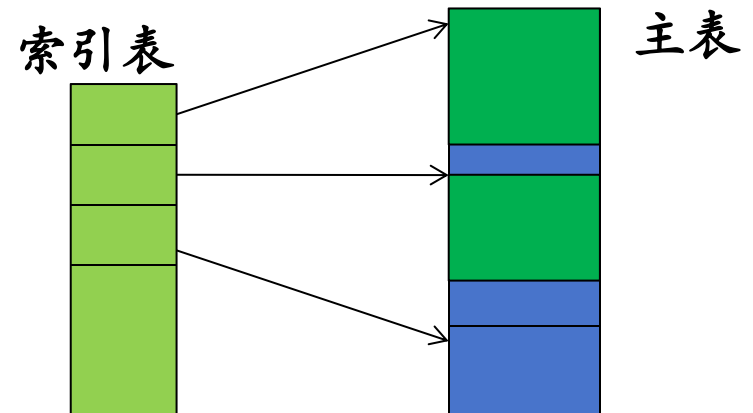
折半查找索引表+顺序查找被确定的块

$$ASL_{bs} = [\log_2(b+1) - 1] + (s+1)/2 \approx \log_2(n/s+1) + s/2$$

分块查找效率介于顺序查找和折半查找之间

[算法特点]

- 实用中，各块大小不一定相等
- 分块查找的代价是附加索引表的存储空间开销和建立索引表的时间开销
- 可以在主表的每块后预留空闲位置，以便进行插入和删除操作时只涉及相应的块和该块的索引项。
- 主表中各块可以集中顺序存储，也可以按块分别顺序存储；主表中的记录还可以采用单链表，或每块组成一个单链表。



9.2 搜索树

9.2.1 二叉排序树

9.2.2 平衡二叉树（补充内容：红黑树）

9.2.3 B-树（平衡的多路查找树）

9.2.4 B+树

9.2.5 键树

9.2.1 二叉排序树BST(二叉查找/搜索树)

1. 定义

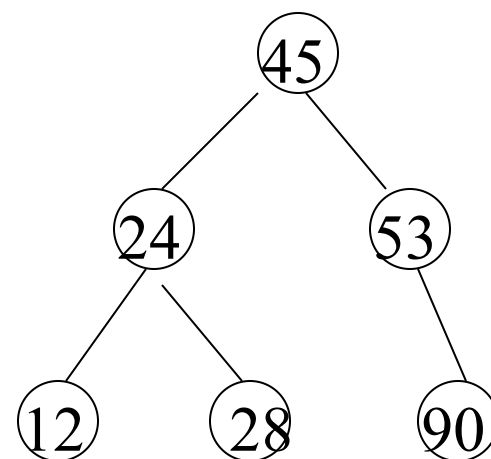
<https://www.cs.usfca.edu/~galles/visualization/BST.html>

二叉排序树或者是空树，或者是满足如下性质的二叉树：

- 1)若其左子树非空，则左子树上所有结点的值均小于根结点的值；
- 2)若其右子树非空，则右子树上所有结点的值均大于根结点的值；
- 3)其左右子树本身又各是一棵二叉排序树

2. 性质

中序遍历一棵二叉排序树，将得到一个以关键字递增排列的有序序列



3. 在二叉排序树上的操作

通常，取二叉链表作为存储结构

1)查找

```
Bitree SearchBST ( BiTree T, KeyType key )
```

```
// 在二叉排序树T中查找关键字值为 key 的结点,
```

```
//找到返回该结点的地址，否则返回空。
```

```
{ if ( (!T) || EQ( key, T ->data. key ) )
```

```
    return ( T ); //T为空或者查找成功
```

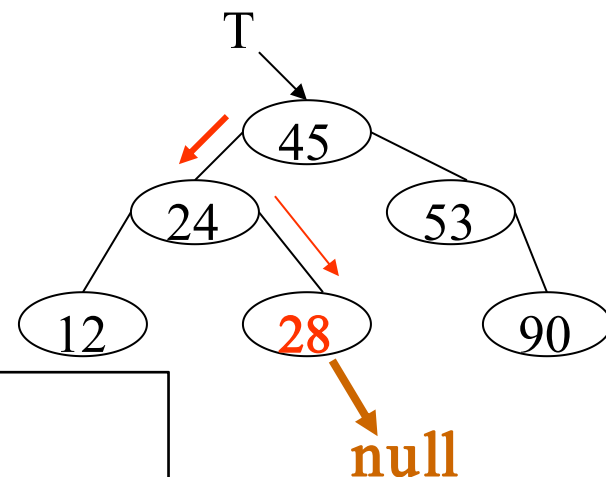
```
else if ( LT( key , T ->data. key ) )
```

```
    return (SearchBST ( T -> lchild, key ));
```

```
else
```

```
    return (SearchBST ( T -> rchild, key ));
```

```
} //SearchBST P228-9.5 (a)
```

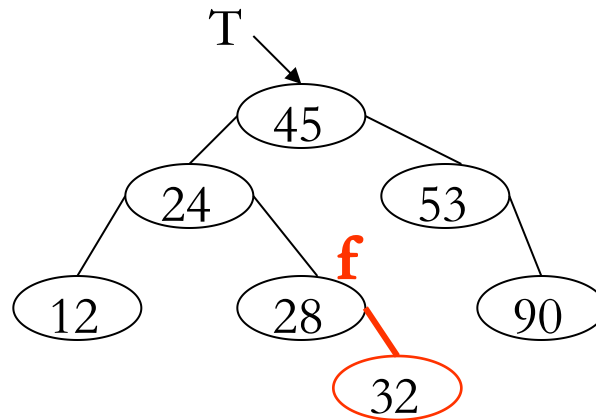


key=28

查找成功

key=32

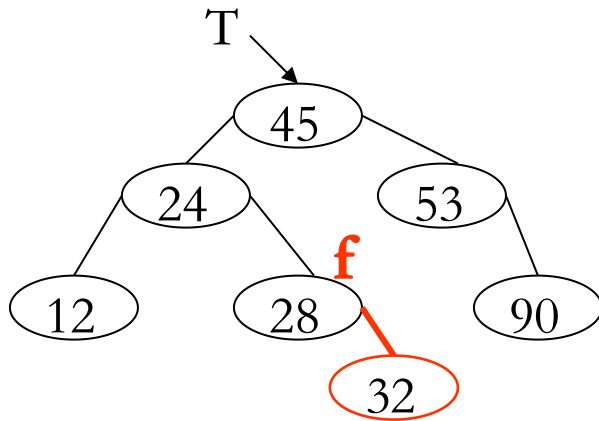
查找失败



```

Status SearchBST(BiTree T, KeyType key, BiTree f, BiTree &p)
//f指向T的父结点,初始调用值为NULL。
//查找成功, p指向该结点, 并返回TRUE;
//查找失败, p指向查找路径上的最后一个结点, 并返回FALSE
{ if (!T) {p=f; return FALSE;} //查找失败, p=f, 即p指向父结点
  else if EQ(key, T->data.key)
    { p=T; return TRUE; } //查找成功, p指向该结点
  else if LT(key, T->data.key)
    return SearchBST(T->lchild, key, T, p );
  else return SearchBST(T->rchild, key, T, p);
} //SearchBST P228-9.5(b)
  
```

2) 插入

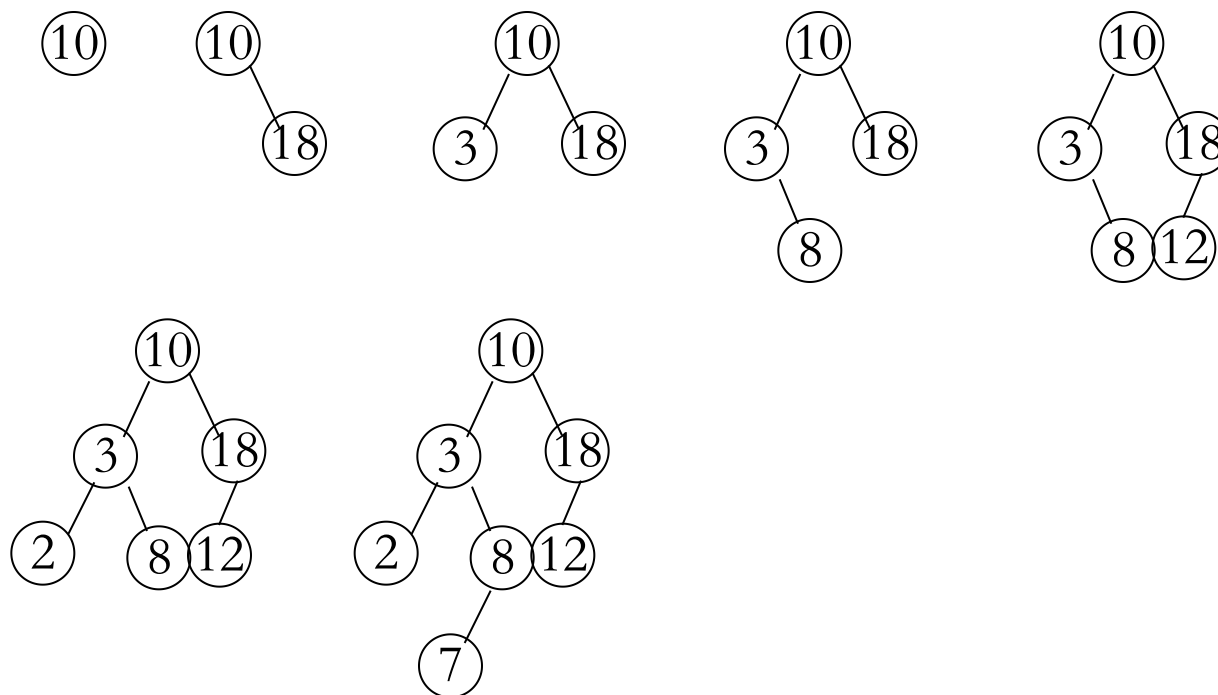


```
Status InsertBST (BiTree &T, ElemType e)
{ if (!SearchBST(T, e.key, NULL, p)) { //未找到
    s=(BiTree)malloc(sizeof(BiTNode));
    s->data=e;
    s->lchild=s->rchild=NULL;
    if (!p) T=s; //若父结点为空, T直接指向s
    else if LT(e.key, p->data.key)
        p->lchild = s; //放p的左侧
    else p->rchild = s; //放p的右侧
    return TRUE;
}
else return FALSE
} //InsertBST P228-9.6
```

3) 生成

从空树出发，反复调用插入操作InsertBST即可。

[例] {10, 18, 3, 8, 12, 2, 7}

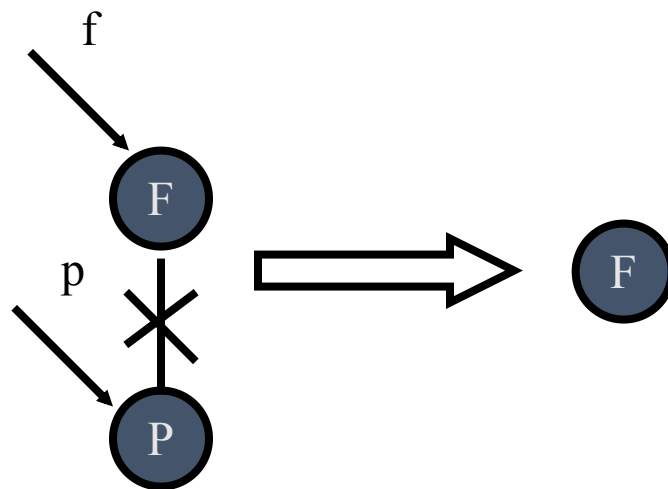
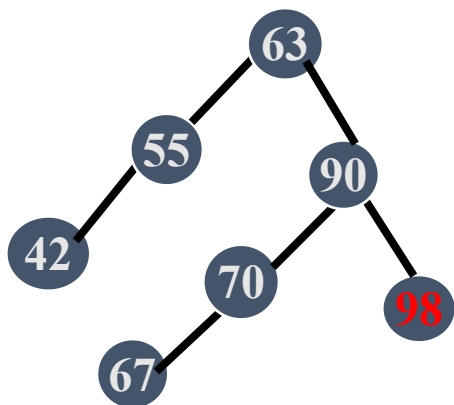


4. 二叉排序树中删除一个结点

设待删结点为 p (p 为指向待删结点的指针)，其双亲结点为 f ，以下分三种情况进行讨论。

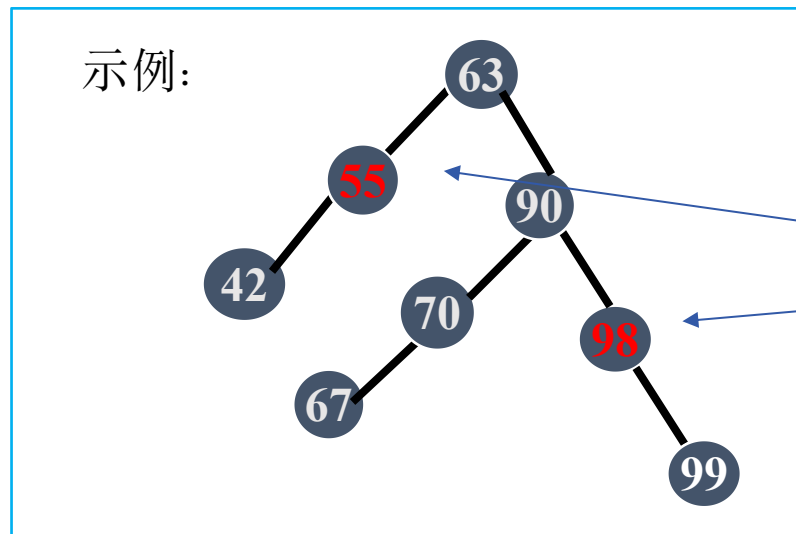
(1) p 结点为叶结点。

示例:



如何删除结点98?

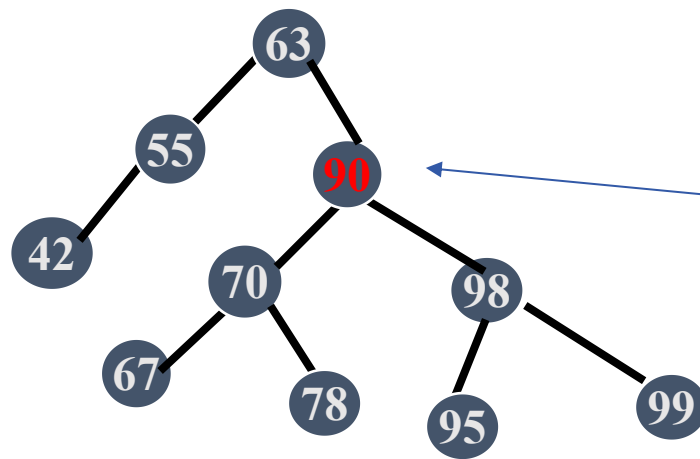
(2) p结点只有右子树 p_R 或只有左子树 p_L 。



如何删除结点55
或结点98?

(3) p结点既有左子树 P_L 又有右子树 P_R ，可按中序遍历保持有序进行调整。

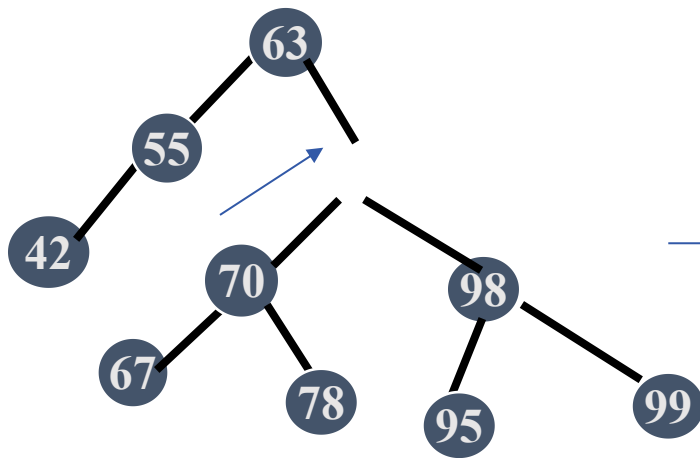
示例：



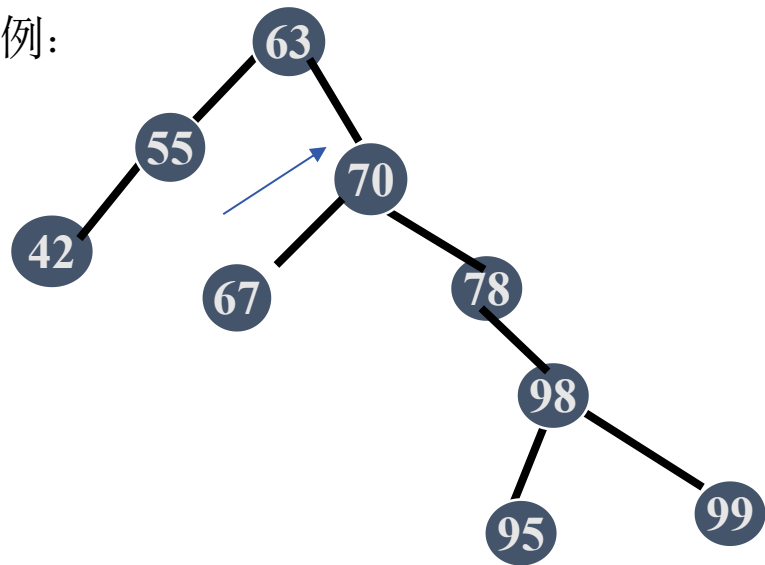
如何删除结点90?

(3) p结点既有左子树 P_L 又有右子树 P_R ，可按中序遍历保持有序进行调整。

示例：



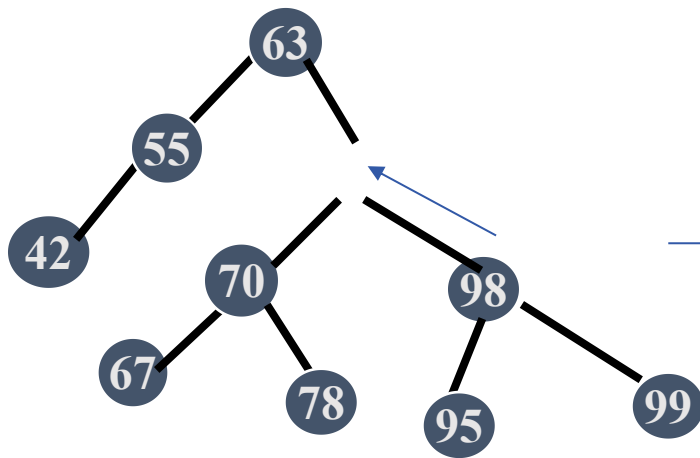
示例：



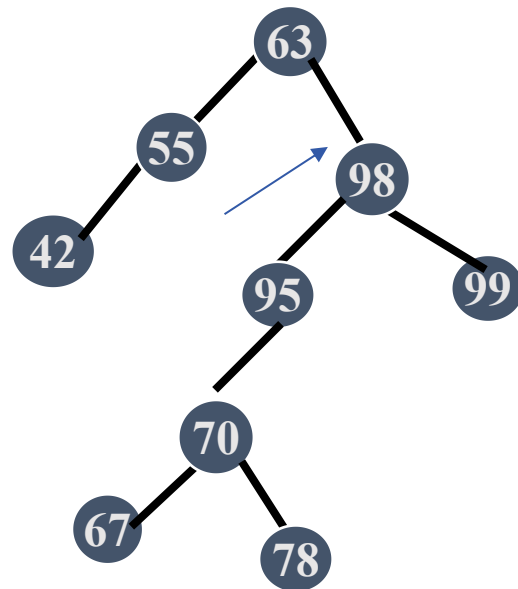
方案一：左子树或右子树跟上去

(3) p结点既有左子树 P_L 又有右子树 P_R ，可按中序遍历保持有序进行调整。

示例：

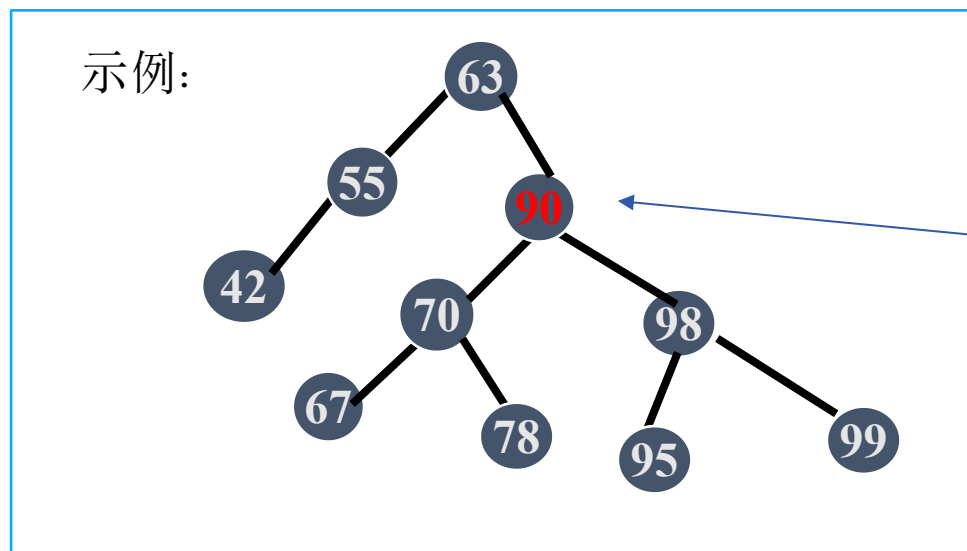


示例：



方案一：左孩子或右孩子跟上去

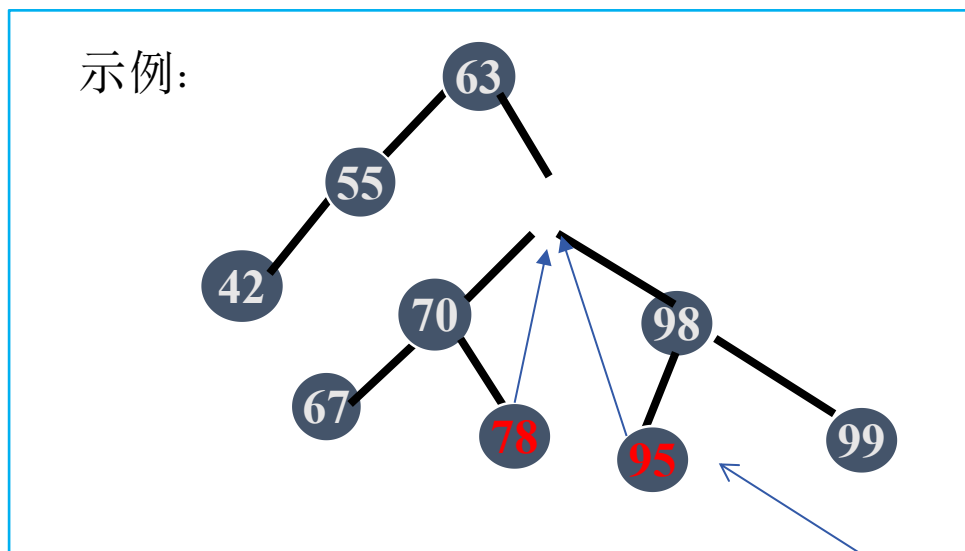
(3) p结点既有左子树 P_L 又有右子树 P_R ，可按中序遍历保持有序进行调整。



如何删除结点90?

方案二：把左子树的最右叶子结点拿上去
或者把右子树的最左叶子结点拿上去

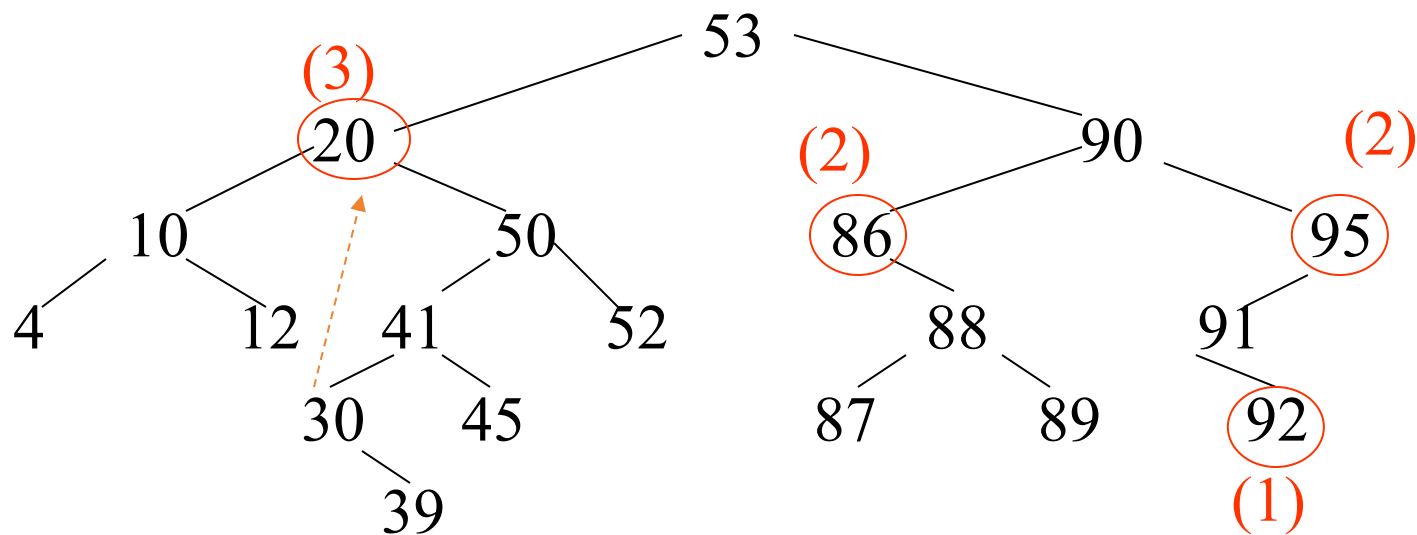
(3) p结点既有左子树 P_L 又有右子树 P_R ，可按中序遍历保持有序进行调整。



如果95有右孩子怎么办？

方案二*：把左子树的最右下叶子（中序前驱）结点拿上去
或者把右子树的最左下叶子（中序后继）结点拿上去

4) 删除



被删除结点为 $*p$ 的不同情况分析:

- (1) p 是叶子结点: 修改其双亲指针即可
- (2) p 只有左孩子: 用 p 的左子树代替以 p 为根的子树
 p 只有右孩子: 用 p 的右子树代替以 p 为根的子树
- (3) p 有两个孩子: 找到 p 的中序后继(或前趋)结点 q ;
 q 的数据复制给 p ;
 删除只有右孩子(或左孩子)/无孩子的 q 。

4) 删除

Status DeleteBST (BiTree &T, KeyType key)

//若二叉排序树T中存在关键字等于key的数据元素时，则删除该结点

//并返回TRUE，否则返回FALSE

{ if(!T) return FALSE; //若T为空，返回FALSE

else {

if (EQ(key, T->data.key)) { return Delete(T) // 若找到则删除

else if (LT(key, T->data.key)) //若待删除的key小于根结点的key

return DeleteBST(T->lchild, key) //去左子树中删除key

else //若待删除的key大于根结点的key

return DeleteBST(T->rchild, key) //去右子树中删除key

}

} //DeleteBST

Status Delete (BiTree &p)

//从二叉排序树中删除结点p，并重接它的左或右子树

{ if(!p->rchild) { //若p的右子树为空，则只需重接它的左子树

 q=p ; p=p->lchild; free(q); //如处理右上图55结点

else if(!p->lchild){ //若p的左子树为空，则只需重接它的右子树

 q=p ; p=p->rchild; free(q); //如处理右上图98结点

} else { //左右子树均不为空 //如删除右下图中的10结点

 q=p; s=p->lchild // s指向p的左子树

 while(s->rchild) { q=s; s=s->rchild} // 找到左子树的右下角

 p->data = s->data; //把左子树的右下角结点的值拿上来

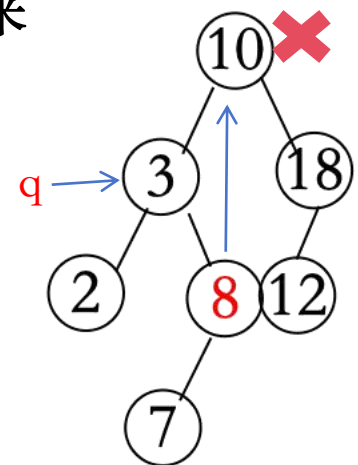
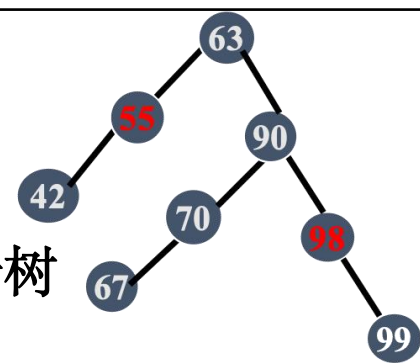
 if(q!=p) q->rchild =s->lchild; //重接*q的右子树

 else q->lchild = s->lchild; //重接*q的左子树

 free(s);

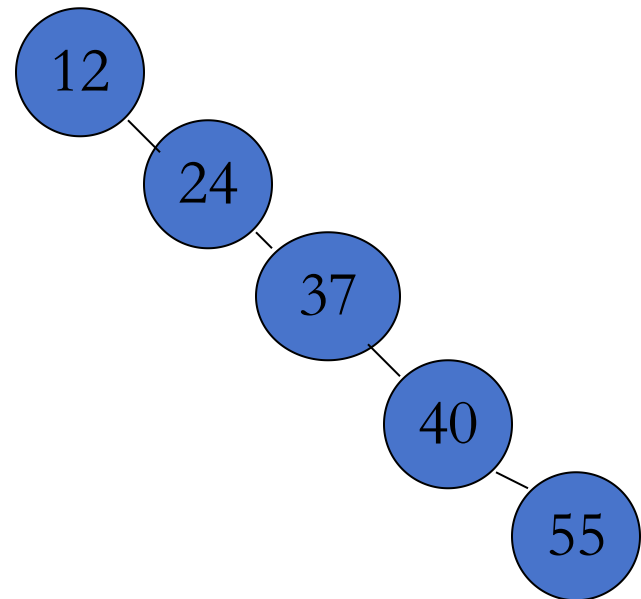
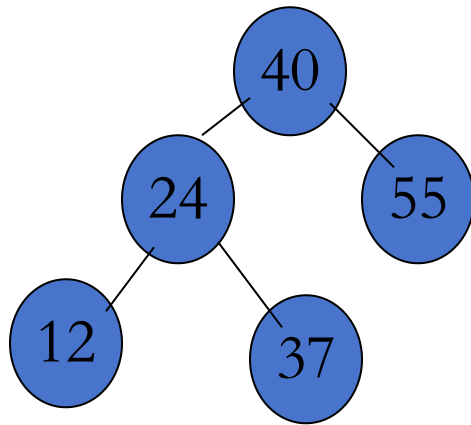
 } return TRUE;

}//Delete



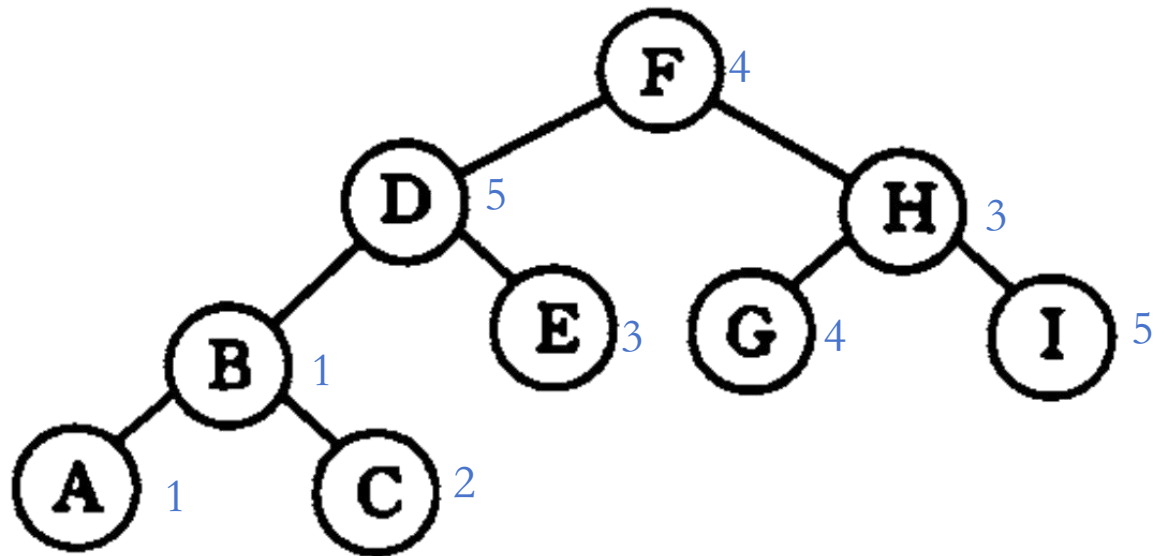
4. 性能分析

- 给定树的形态，等概率查找成功时的 $ASL = \sum c_i / n$
 - 最差(单支树): $(n+1)/2$
 - 最好(类似折半查找的判定树): $\log_2(n+1)-1$
 - 随机: $\leq 1 + 4\log_2 n$
- 关键字有序出现时，构造出“退化”的二叉排序树，树深为 n ，各种操作代价 $O(n)$ 。避免方法：采用平衡二叉树



最优/次优查找树与二叉排序树

- 最优/次优查找树也符合二叉排序树的定义
- 二叉排序树的性能分析的讨论是在等概率前提下进行的
- 最优/次优查找树考虑了查找概率不等的问题
- 最优/次优查找树是静态树表，不方便在查找的过程中进行插入/删除
- 二叉排序树是动态树表，方便在查找的过程中进行插入/删除



9.2.2 平衡二叉树(AVL树)

1.平衡二叉树的定义

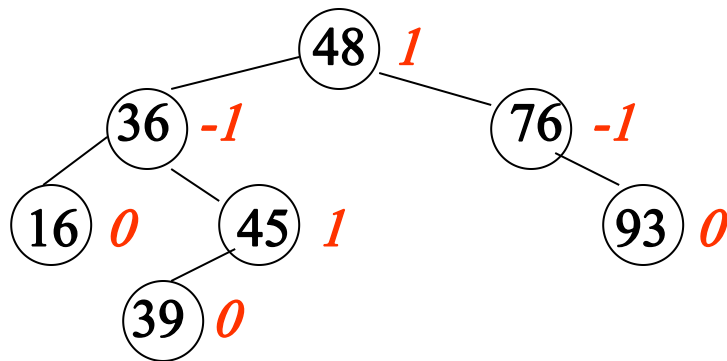
或者是空树，或者是满足如下性质的**二叉排序树**：

1)它的左、右子树的**高度之差**的**绝对值不超过1**；

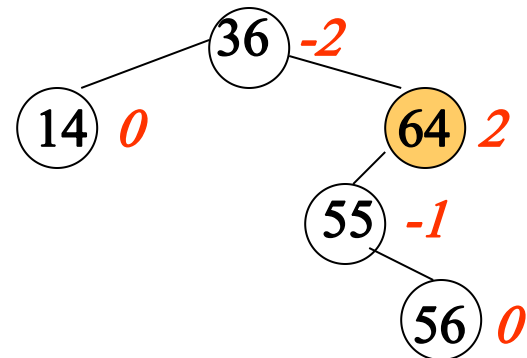
$$|h_L - h_R| \leq 1$$

2)其左右子树本身又各是一棵平衡二叉树。

二叉树上结点的**平衡因子**：该结点的**左子树**高度**减去右子树**的高度。



平衡二叉树



非平衡二叉树

平衡二叉树不一定是理想的最矮状态。

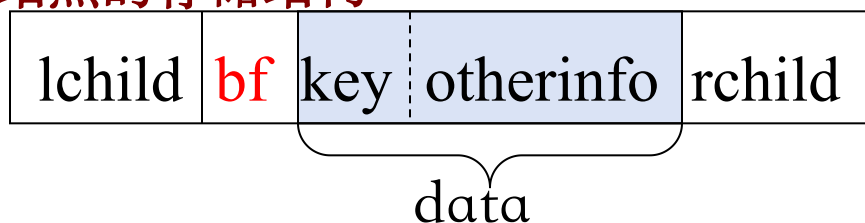
2. 定理

一个具有n个结点的平衡二叉树的高度h为

$$\log_2(n+1) \leq h \leq \log_a(\text{sqrt}(5)*(n+1))-2 \quad a=(1 + \text{sqrt}(5))/2$$

结论：最坏情况下，AVL树的高度约为 $1.44\log_2 n$ ，而完全平衡的二叉树高度约为 $\log_2 n$ ，因此AVL树是接近最优的，其平均查找长度与 $\log_2 n$ 同数量级。

3. 结点的存储结构



lchild: 左孩子指针

rchild: 右孩子指针

bf: 平衡因子

key: 记录的关键字

```
typedef struct {
    KeyType key;
    .....
} ElemType
typedef struct BSTNode {
    ElemType data;
    int bf;
    struct BSTNode *lchile, *rchild;
} BSTNode, * BSTree;
```

4. 在平衡二叉树上的操作

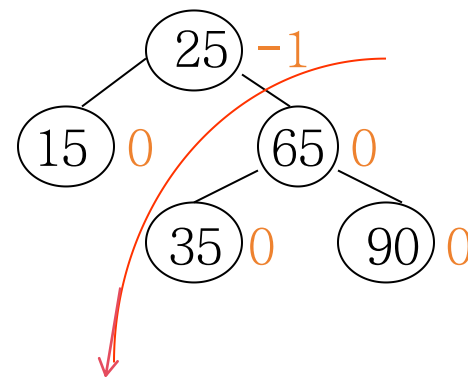
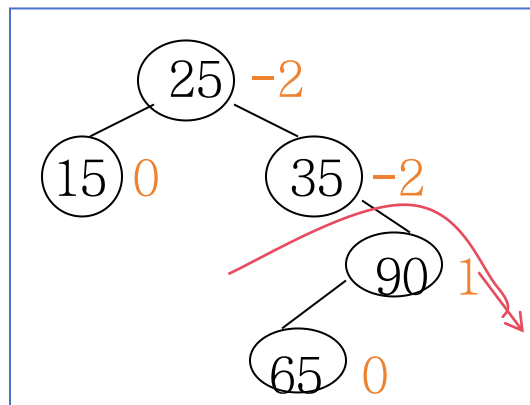
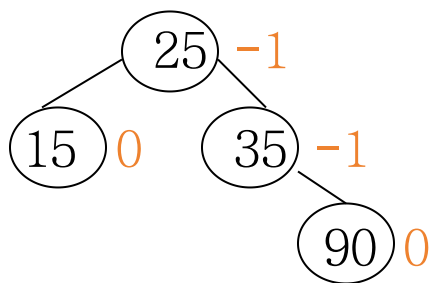
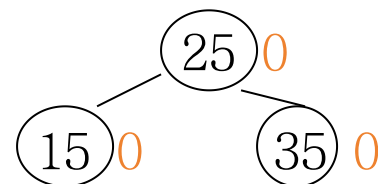
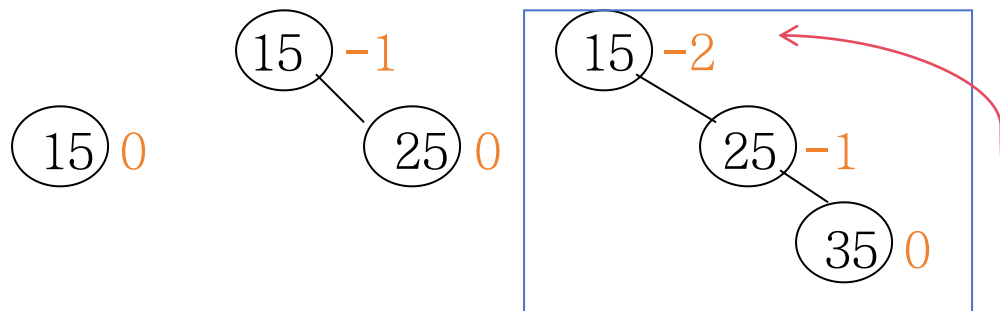
1) 查找

查找方法同二叉排序树。

2) 插入 示例序列：15、25、35、90、65

[例] 平衡二叉树的生成过程

在插入过程中采用平衡旋转技术

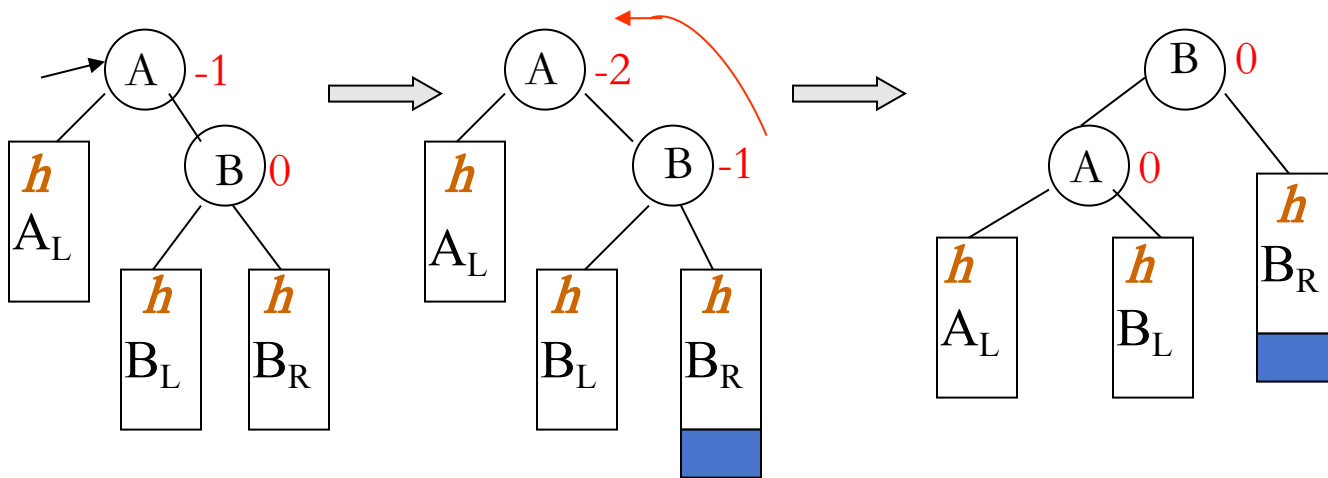


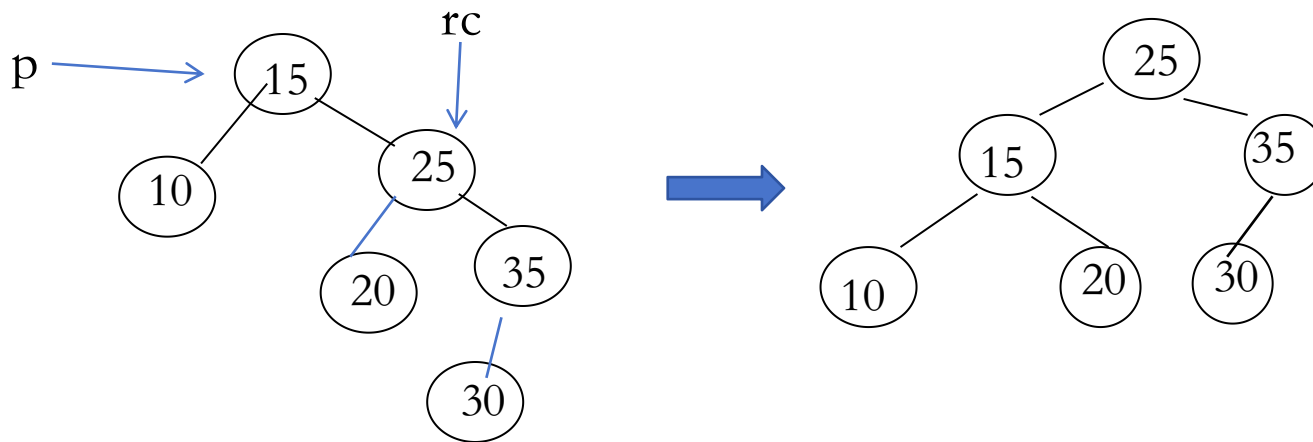
[调整范围的确定]

插入结点后，找到离插入结点最近且平衡因子绝对值超过1的祖先结点，则以该结点为根的子树将是可能不平衡的**最小子树**，可将重新平衡的范围局限于这棵子树。

[调整的规律] 设失去平衡的最小子树的根结点指针为a

RR型平衡旋转——单向左旋（一次逆时针旋转） “右” 上再加 “右”





```
Status L_Rotate (BSTree &p) {
```

```
//对以*p为根的二叉排序树作左旋处理， 处理之后p指向新的树根结点，  
//即旋转之前的右子树的根结点
```

```
    rc=p->rchild; //rc 指向的*p的右子树根结点
```

```
    p->rchild = rc->lchild; //rc 的左子树挂接为*p的右子树
```

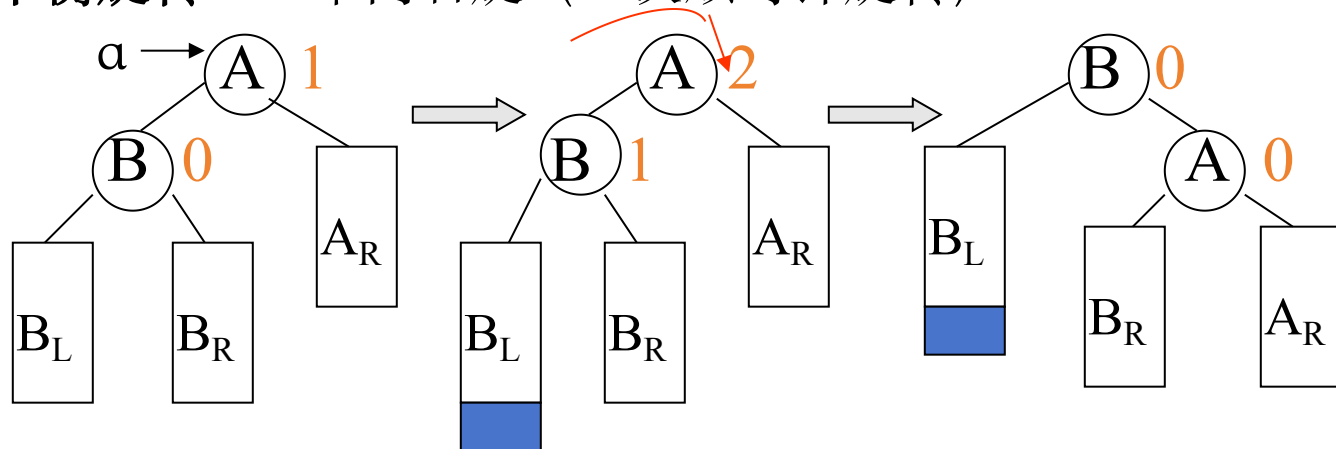
```
    rc->lchild =p; //p挂载为rc的左子树
```

```
    p=rc; //p指向新的根结点
```

```
}//L_Rotate
```

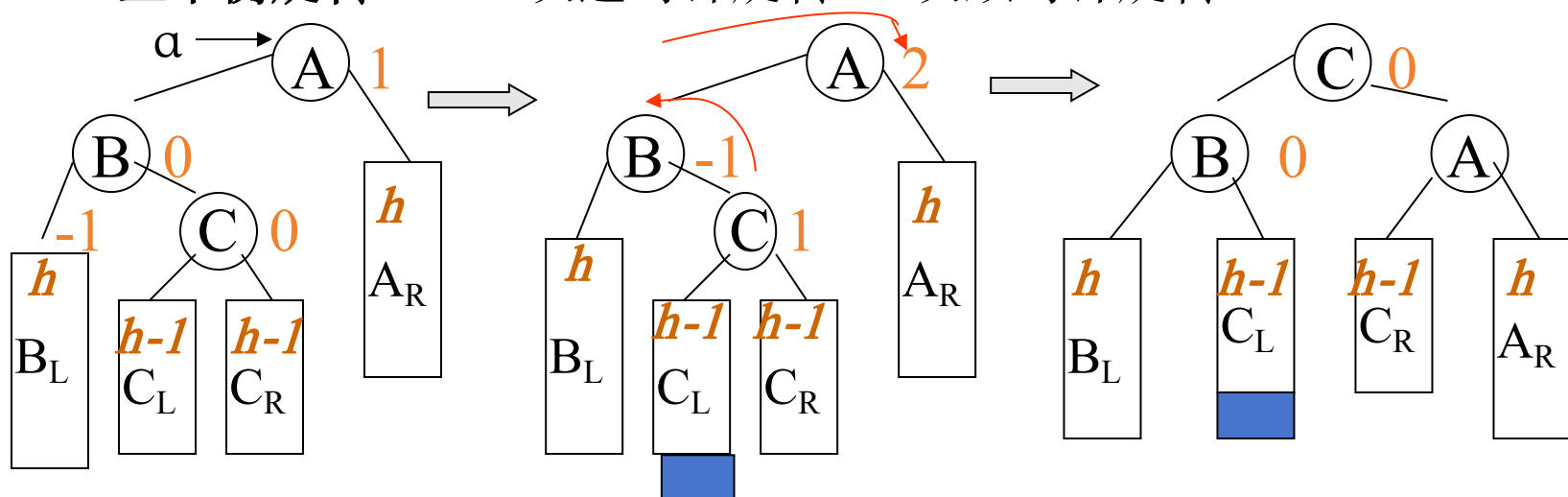
“左”上再加“左”

- LL型平衡旋转——单向右旋（一次顺时针旋转）



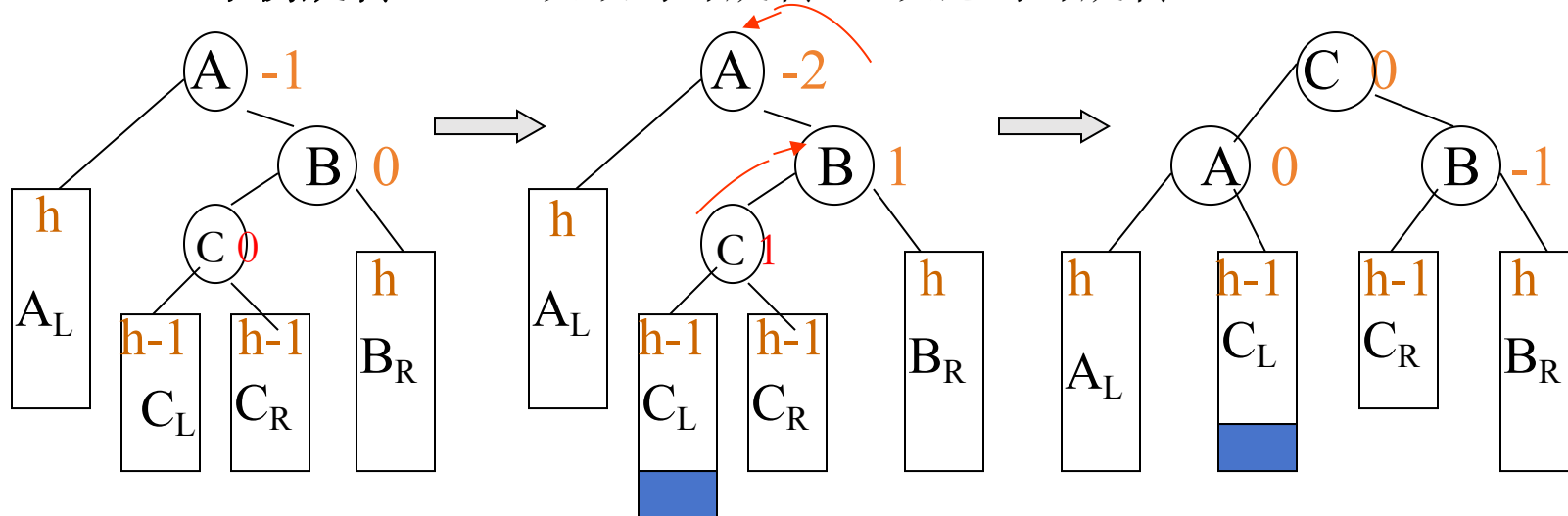
- LR型平衡旋转——一次逆时针旋转+一次顺时针旋转

“左”上加“右”



“右”上加“左”

- RL型平衡旋转——一次顺时针旋转+一次逆时针旋转



[例]

3)删除 (思路同插入)

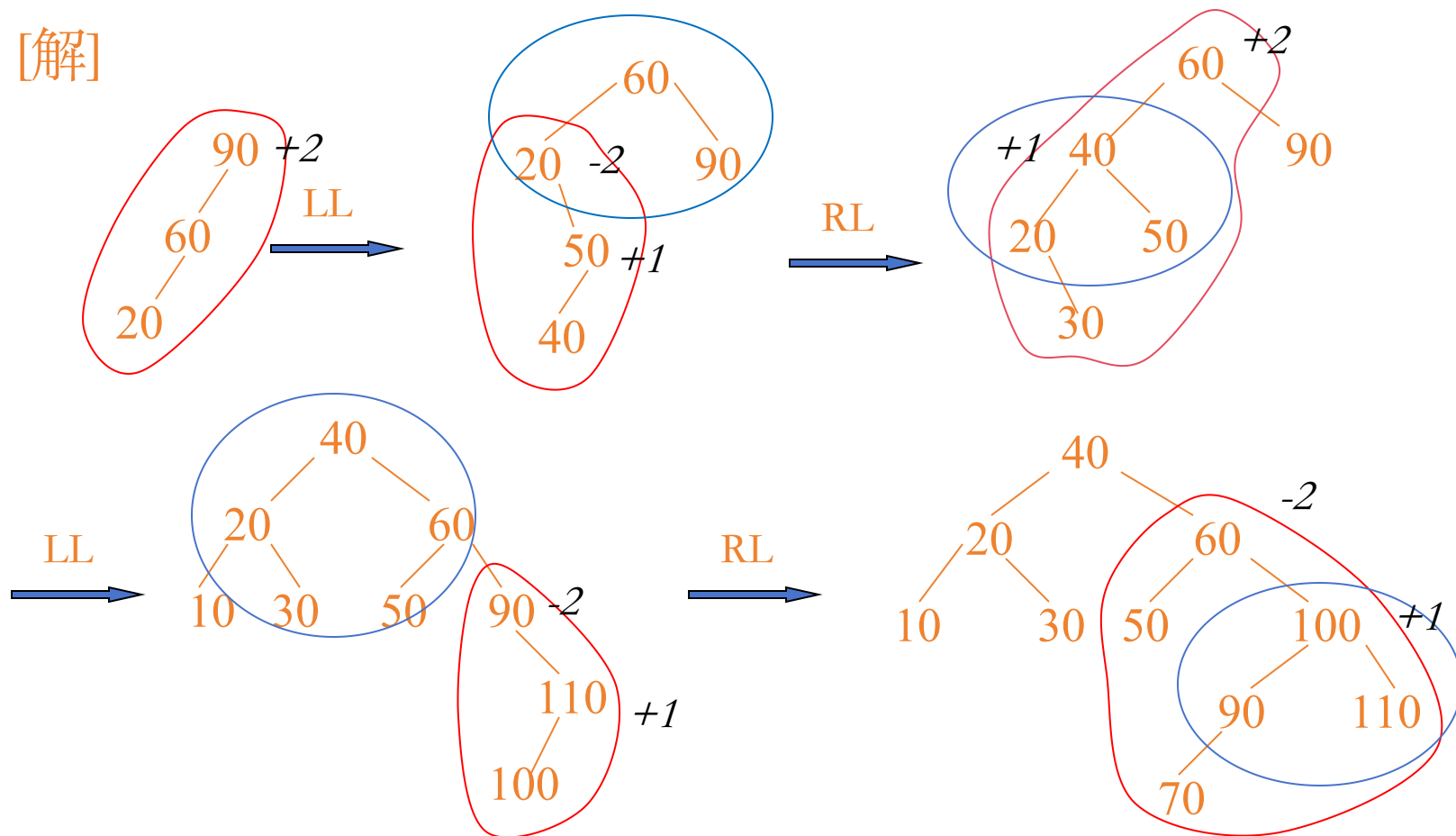
将删除结点 q 转化为 q 最多有一个孩子的情形，即若 q 有两个孩子，则以其中序前驱/后继结点 r 取代它，删除 r ；

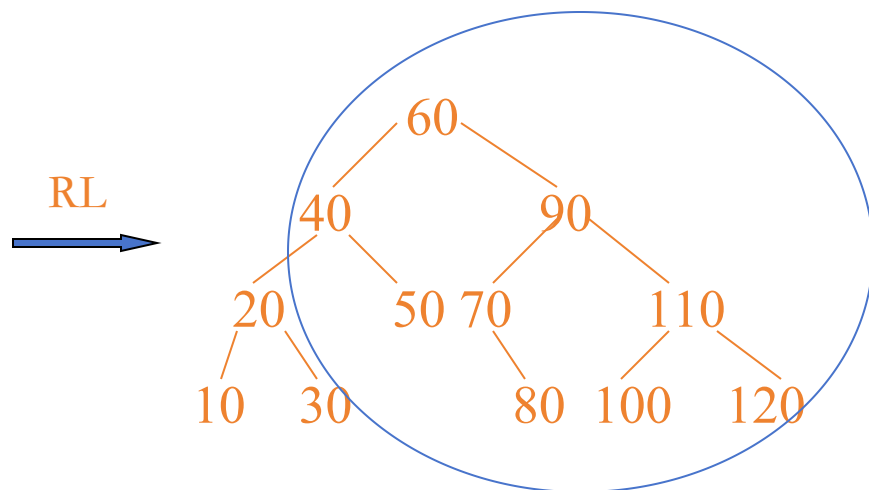
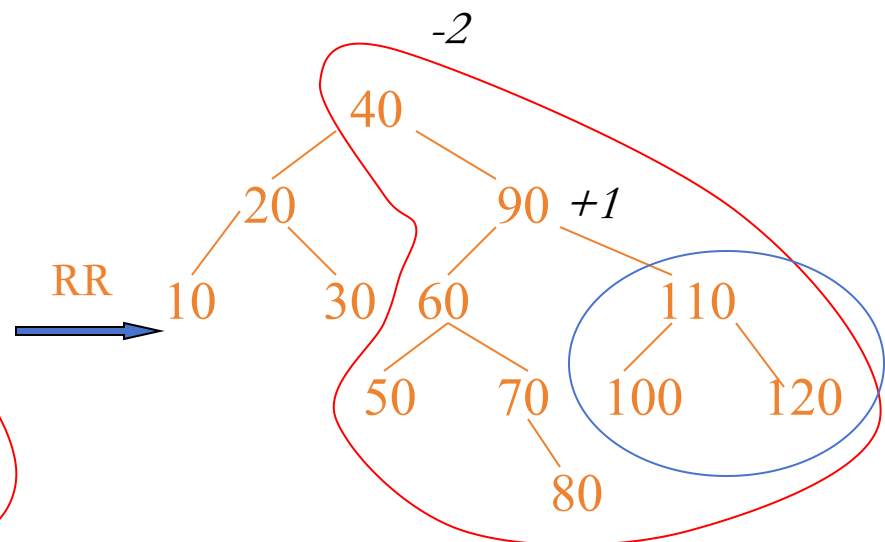
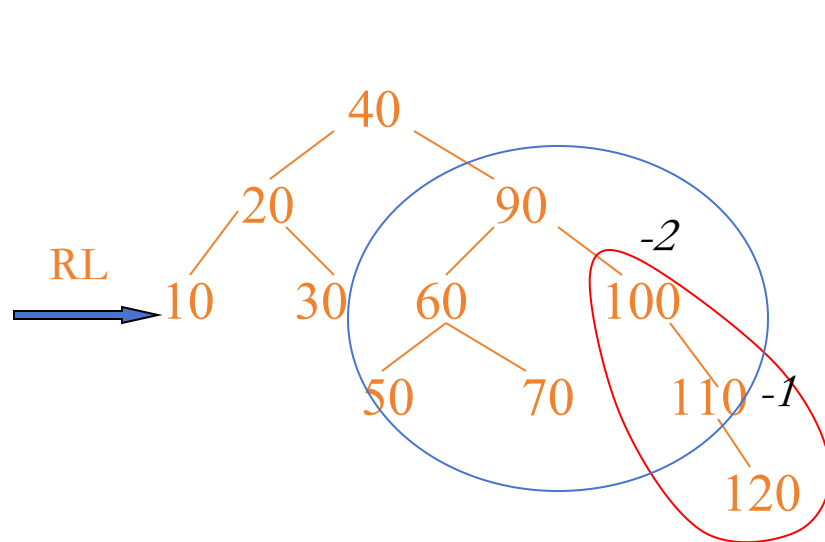
若树的平衡性被破坏，利用单一/双重旋转恢复。

以下表顺序建立平衡二叉排序树，并求在等概率情况下查找成功的平均查找长度。（90,60,20,50,40,30,10,110,100,70,120,80）

红色圈表示要调整的部分，蓝色圈表示调整后

[解]



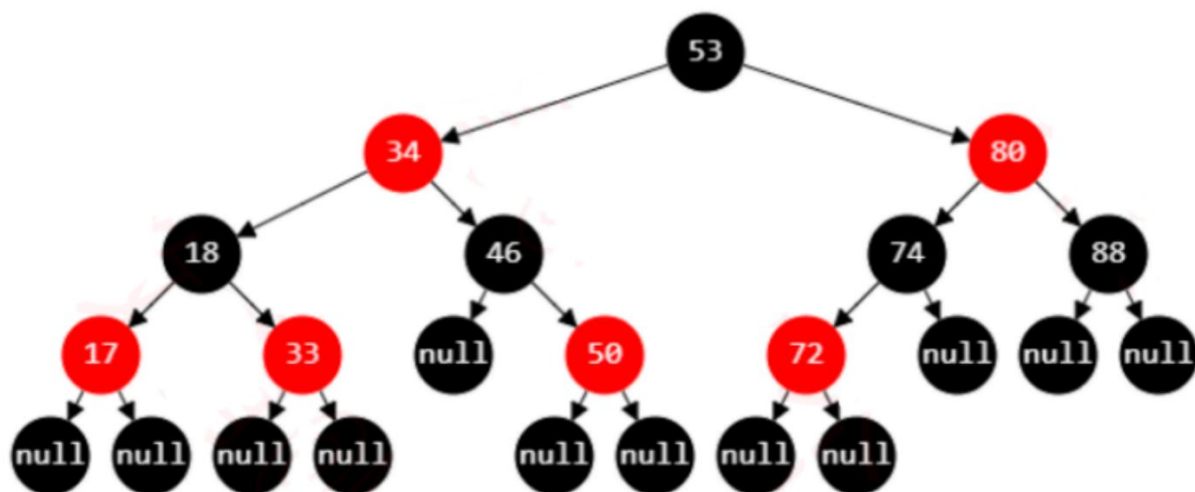


$$ASL = (1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) / 12$$

$$= 37 / 12$$

红黑树 (Red-Black Tree, R-B Tree)

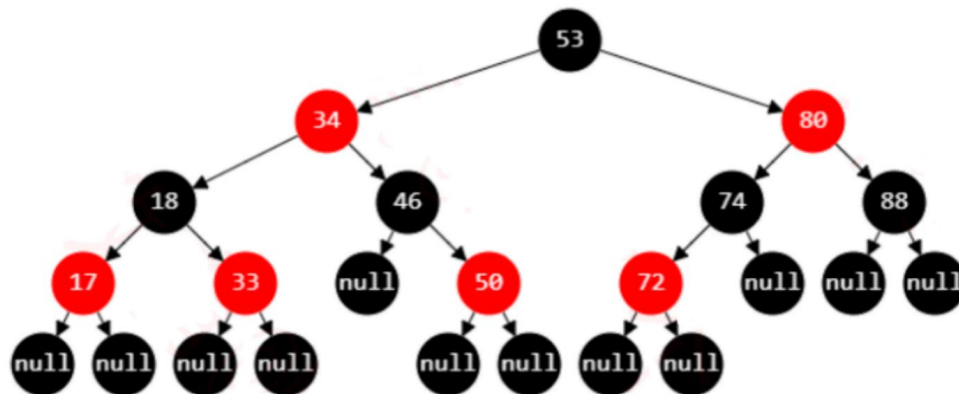
- 红黑树是一种**相对平衡**的二叉查找树，**牺牲了部分平衡性**以换取插入/删除操作时少量的旋转操作
- 红黑树的高度近似为 $\log_2 n$ ，插入、删除和查找操作的时间复杂度都是 $O(\log_2 n)$
- Java的TreeMap，C++ STL的map等很多都基于红黑树实现。红黑树



红黑树 (Red-Black Tree, R-B Tree)

红黑树需要满足以下要求:

- 1) 结点只能是红色或黑色的;
- 2) 根结点是黑色的;
- 3) 每个叶子结点都是黑色的空结点;
- 4) 每个红色结点的两个子结点都是黑色,
也就是说红色结点被黑色结点隔开;
- 5) 对于每个结点, 从该结点到其叶子结点
的所有路径, 都包含相同数目的黑色结点。

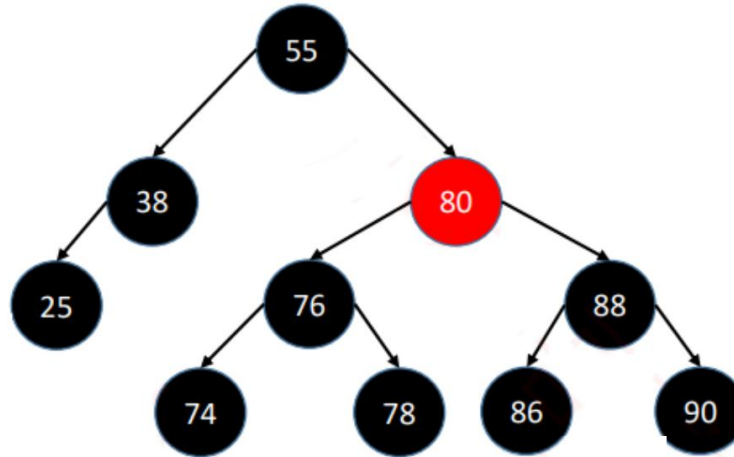


通过染色、旋转调平

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

****从根到叶子的最长的可能路径不多于最短的可能路径的两倍长****

红黑树 (Red-Black Tree, R-B Tree)



上图不是红黑树，因为红黑树需要补充上值为空的黑色叶子结点，上图补充值为空的叶子结点后，有从某个结点出发到其叶子结点的路径上黑色结点数目不等的情况

通过染色、旋转调平

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

9.2.3 B-树 (B-tree, B即balanced)

<https://www.cs.usfca.edu/~galles/visualization/BTree.html>

1. 定义 B-树是一种平衡的多路查找树

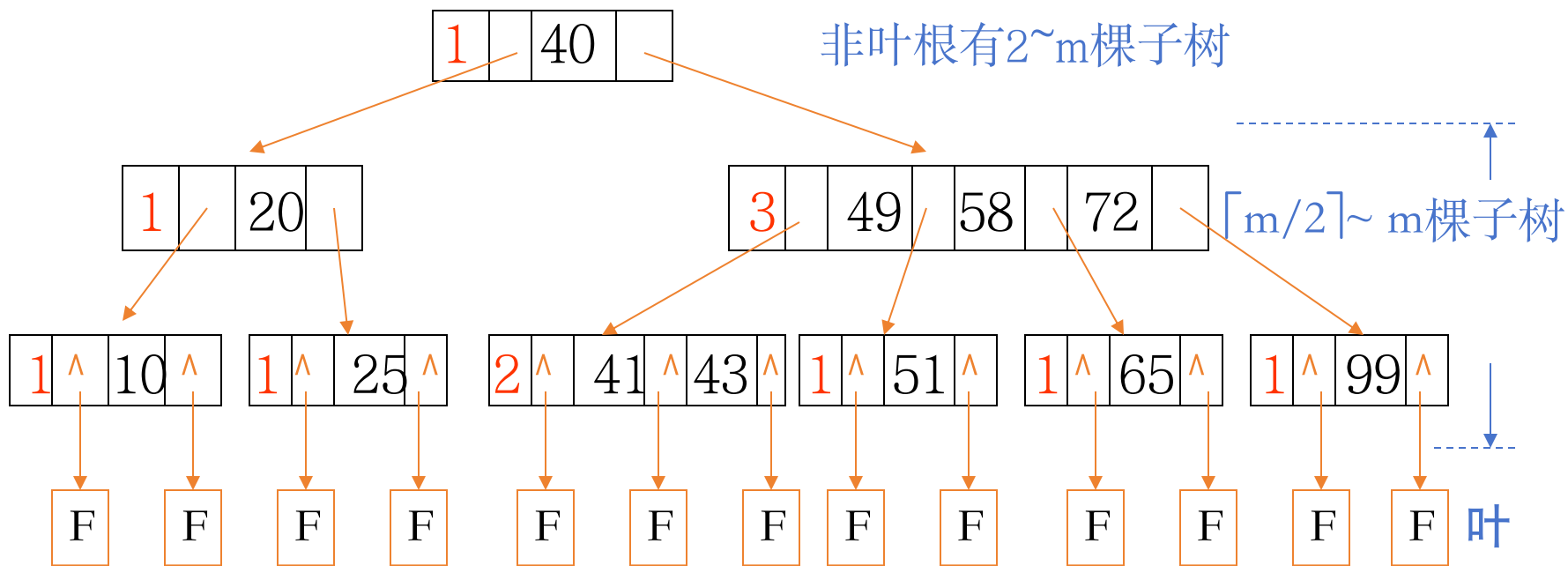
一棵 m 阶B-树，或为空树，或为满足下列特性的 m 叉树：

- 1) 树中每个结点最多有 m 棵子树；（最多有 $m-1$ 个关键字）
- 2) 若根结点不是叶子结点，则最少有两棵子树；
- 3) 除根之外的所有非叶子结点最少有 $\lceil m/2 \rceil$ 棵子树；（最少有 $\lceil m/2 \rceil - 1$ 个关键字）
- 4) 所有非叶子结点包含 $(n, A_0, K_1, A_1, K_2, \dots, K_n, A_n)$ 信息数据；其中 n 为结点中关键字个数， A_i 为指向子树的指针， K_i 为关键字。
- 5) 所有叶子结点在同一层上，且不带信息。

A_{i-1} 指针所指子树中所有结点关键字小于 K_i , A_i 指针所指子树所有结点大于 K_i

非叶结点结构 $(n, A_0, K_1, A_1, K_2, A_2, \dots, K_n, A_n)$

[例] 一棵4阶B-树 ($m=4$) (又称2-3-4树)



$K=43$ (查找成功)

$K=60$ (查找失败)

2. B-树的用途

当内存中不可能容纳所有数据记录时，在磁盘等直接存取设备上组织动态的查找表。

B-树的根结点可以始终置于内存中；

其余非叶结点放置在外存上，**每一结点**可作为一个读取单位（页/块）

选取较大的阶次 m ，降低树的高度，减少外存访问次数

B-树的信息组织方法

- 每个记录的其它信息与关键字一起存储。查到关键字即可获得记录的完整信息。
- 将记录的外存地址(页指针)与关键字一起存储。查到关键字时，还需根据该页指针访问外存。

3. B-树的存储定义

B-树节点定义

```
#define m 3           //B-树的阶

typedef struct BTreeNode {
    int keynum;         //关键字个数
    struct BTreeNode *parent; //双亲指针
    KeyType key[m+1]; //0号未用
    struct BTreeNode *ptr[m+1];
    Record *recptr[m+1]; //0号未用
} BTreeNode, *BTree;
```

查找结果描述

```
typedef struct {
    BTreeNode *pt; //指向找到的结点
    int i;         //1..m, 结点内序号
    int tag;        //1成功, 0失败
} Result;
```

4. B-树上的基本运算

1)查找

[算法思想]

设查找时给定值为K，从根结点开始

(a)在**当前结点*p**中顺序或者折半查找

若找到某个i，满足 $key[i]=K$ ，则查找成功，返回**p、i、1(成功)**

否则，确定满足 $key[i]<K<key[i+1]$ 的i值

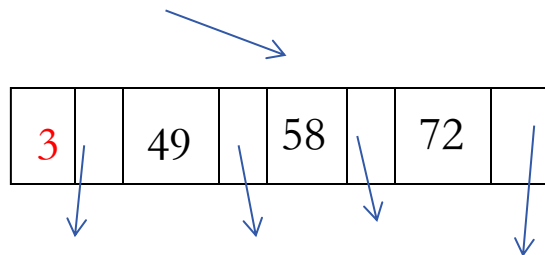
(b)若*p结点中的 $ptr[i]$ 为空指针，则查找失败，返回p、i、0

(c)从磁盘上读入 $ptr[i]$ 指示的结点 **DiskRead(p->ptr[i])**，将此结点作为当前结点，转(a)

[例]

K=43 (查找成功)

K=60 (查找失败)



[算法描述]

Result SearchBTree(BTree T, KeyType K)

{ p = T; q = NULL; found = FALSE; i = 0; //q指向p的双亲, 找不到时q有用

while (p && ! found) {

i = Search(p, K); //在p->key[1..keynum]中找i,
// 使得key[i]≤K<key[i+1]

If (i>0 && p->key[i]==K) found = TRUE;

else { q = p; p = p->ptr[i]; }

}

if (found)

return (p,i,1);

else

return (q,i,0);

}//SearchBTree

2) 插入

[算法思想] 设插入的关键字为K

(a)在B-树中查找K, 若查找到, 则返回

否则查找失败, 查找操作定位于某个最低层的非叶结点(q,i), 在相应的位置插入偶对[K, ^]

(b)keynum加1, 若当前结点未溢出(keynum<m),

则写盘DiskWrite(*q), 返回

(c)以key[$\lceil m/2 \rceil$]为划分点, 分裂当前结点为两个结点:

$(\lceil m/2 \rceil - 1, ptr_0, key_1, \dots, key_{\lceil m/2 \rceil - 1}, ptr_{\lceil m/2 \rceil - 1})$ ——q

$(m - \lceil m/2 \rceil, ptr_{\lceil m/2 \rceil}, key_{\lceil m/2 \rceil + 1}, \dots, key_m, ptr_m)$ ——新结点q1

写盘DiskWrite(*q), DiskWrite(*q1),

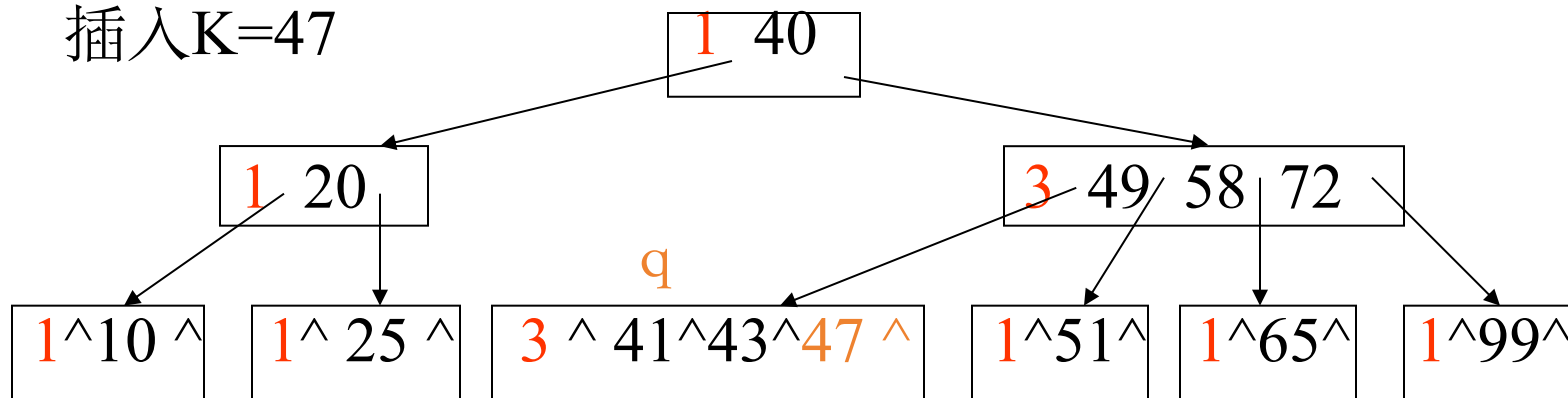
(d)读当前结点的父结点到内存DiskRead(q->parent),

将偶对(key[$\lceil m/2 \rceil$], q1)插入该父结点,

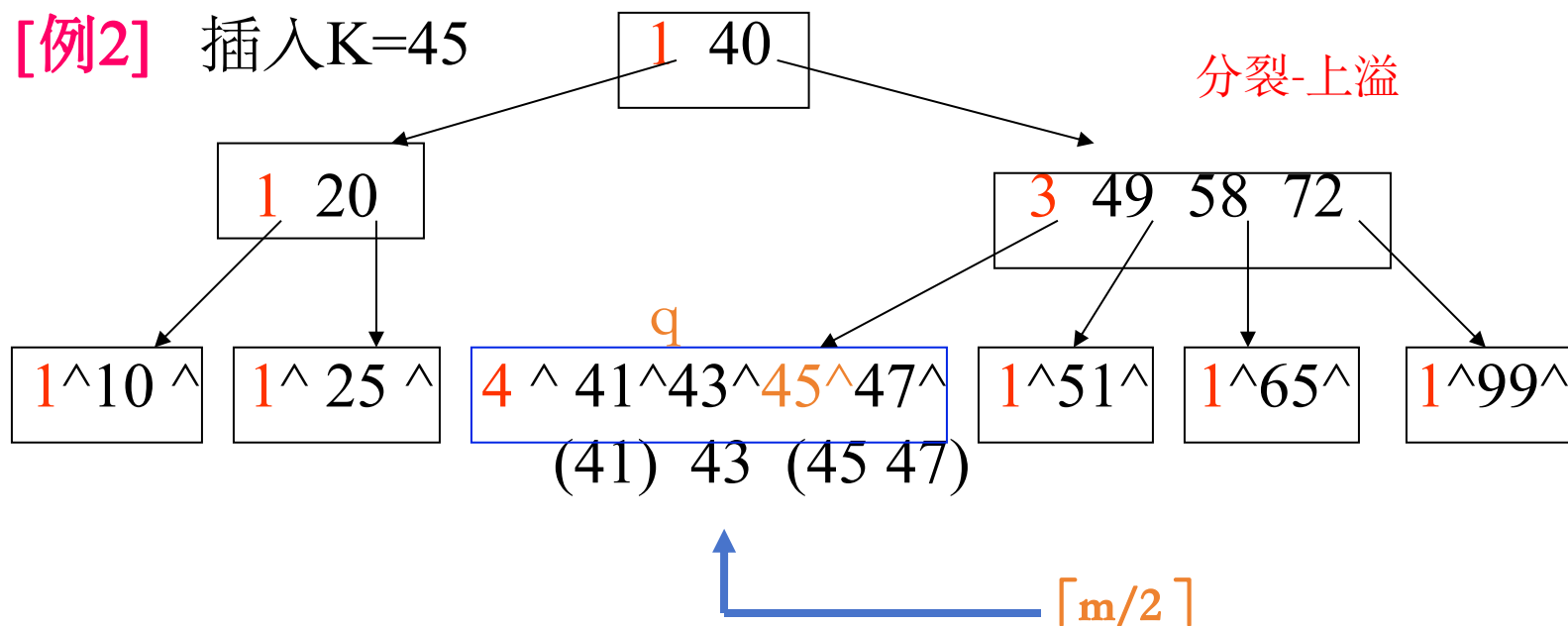
以该父结点为当前结点q, 转(b)

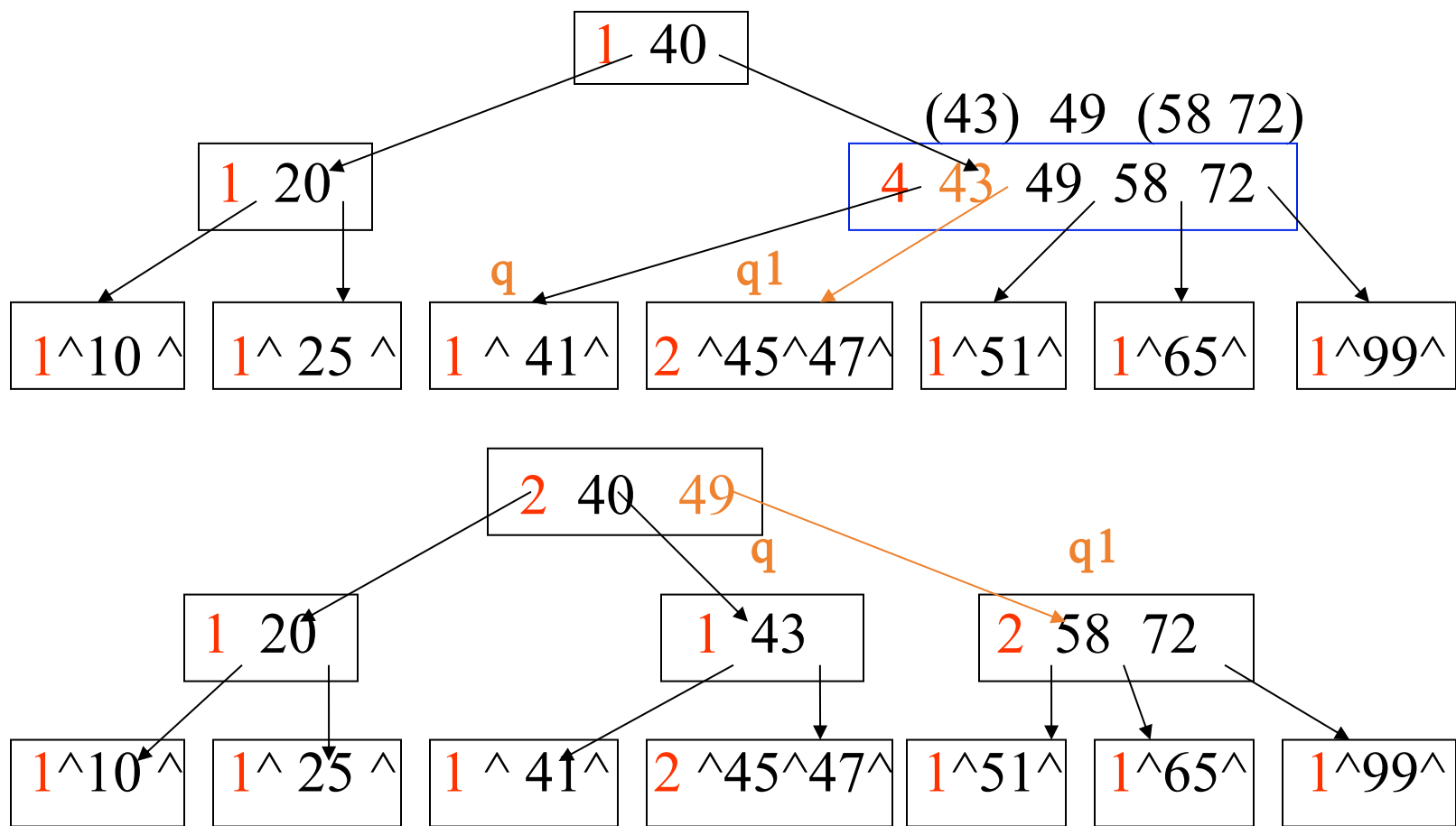
[例1] $m=4$ 的B-树，即分支2-3-4，关键字数量1-2-3

插入K=47



[例2] 插入K=45





若由底向上至根结点仍需分裂时，需要构造新的根结点，B-树会增加一层。

3) 生成

[算法思想]

从空树起，逐个插入关键字得到。

4) 删除

[算法思想]

设删除的关键字为K

(a)在B-树中查找K，若未查找到，则返回

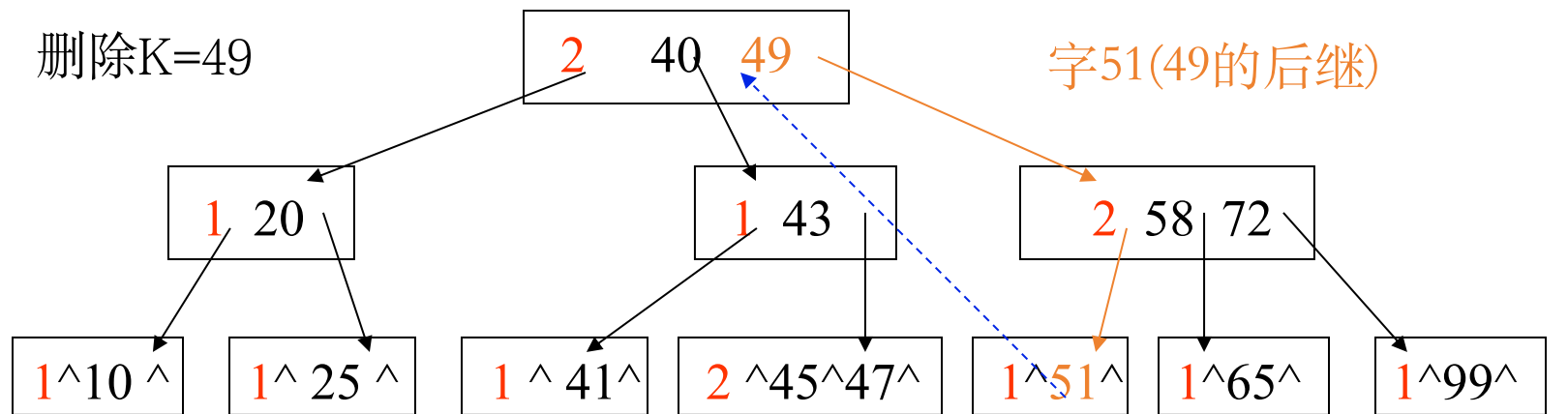
否则查找成功，查找操作**定位于某个结点q**,

且 $q \rightarrow \text{key}[i]$ 为K

(b)若q结点不是最下层的非叶结点，则查找 $q \rightarrow \text{ptr}[i]$ 所指子树的最小关键字x,

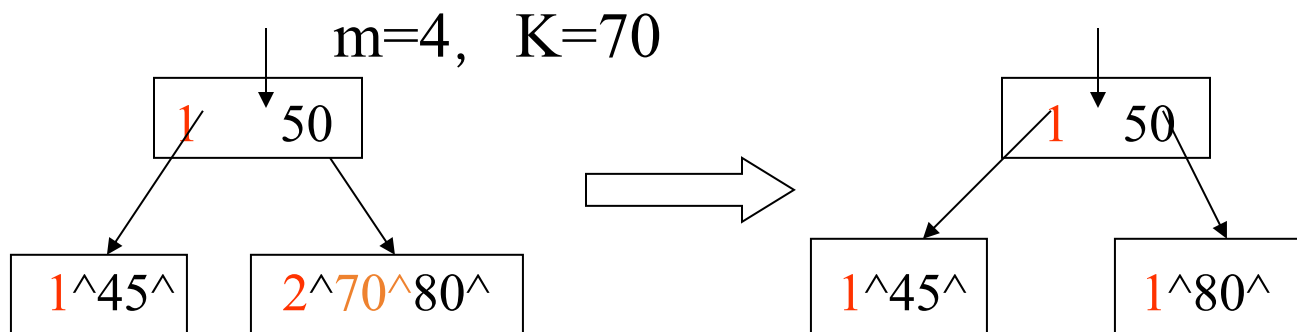
用x替换 $\text{key}[i]$ ，使问题转换为删除**最下层的非叶结点上的关键字**

删除K=49



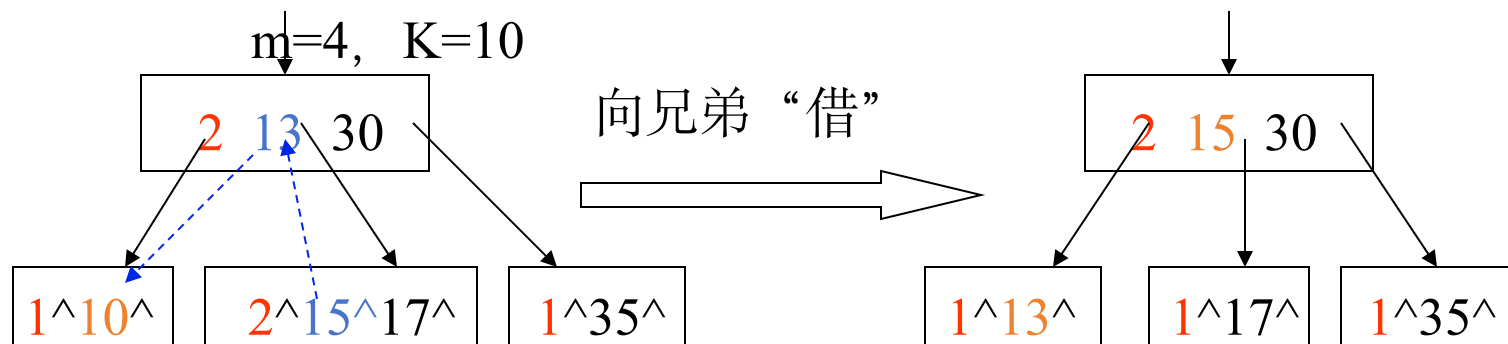
(c) 删除最下层非叶子结点中的关键字，分三种情况处理：

其所属结点的关键字个数 $> \lceil m/2 \rceil - 1$ ：简单删除



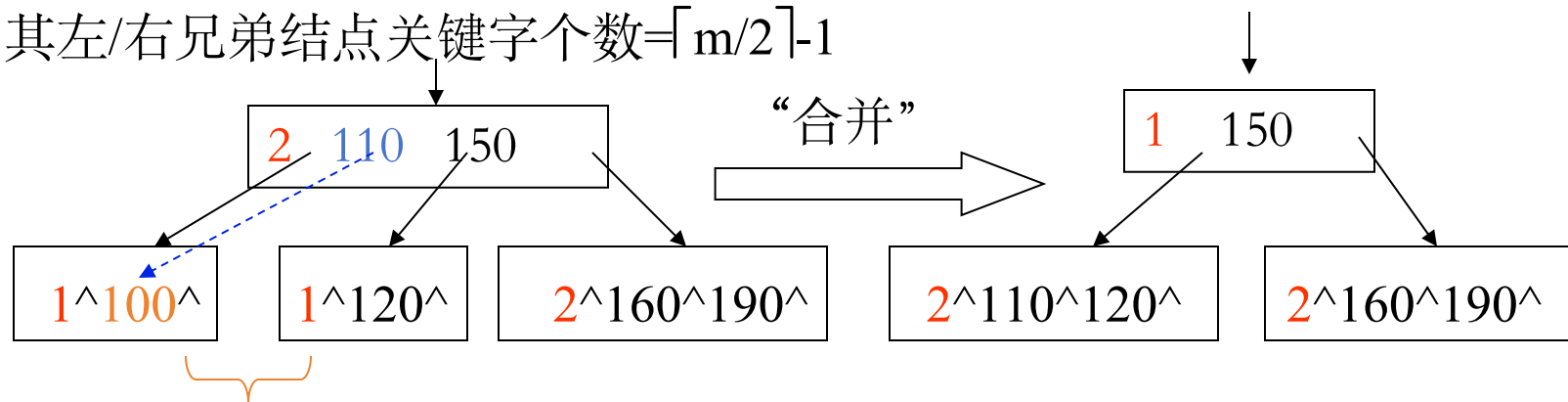
- 其所属结点的关键字个数= $\lceil m/2 \rceil - 1$ (自己不够, 兄弟够),
其左/右兄弟结点关键字个数 $> \lceil m/2 \rceil - 1$

自己 $\leftarrow$ 父结点 $\leftarrow$ 兄弟



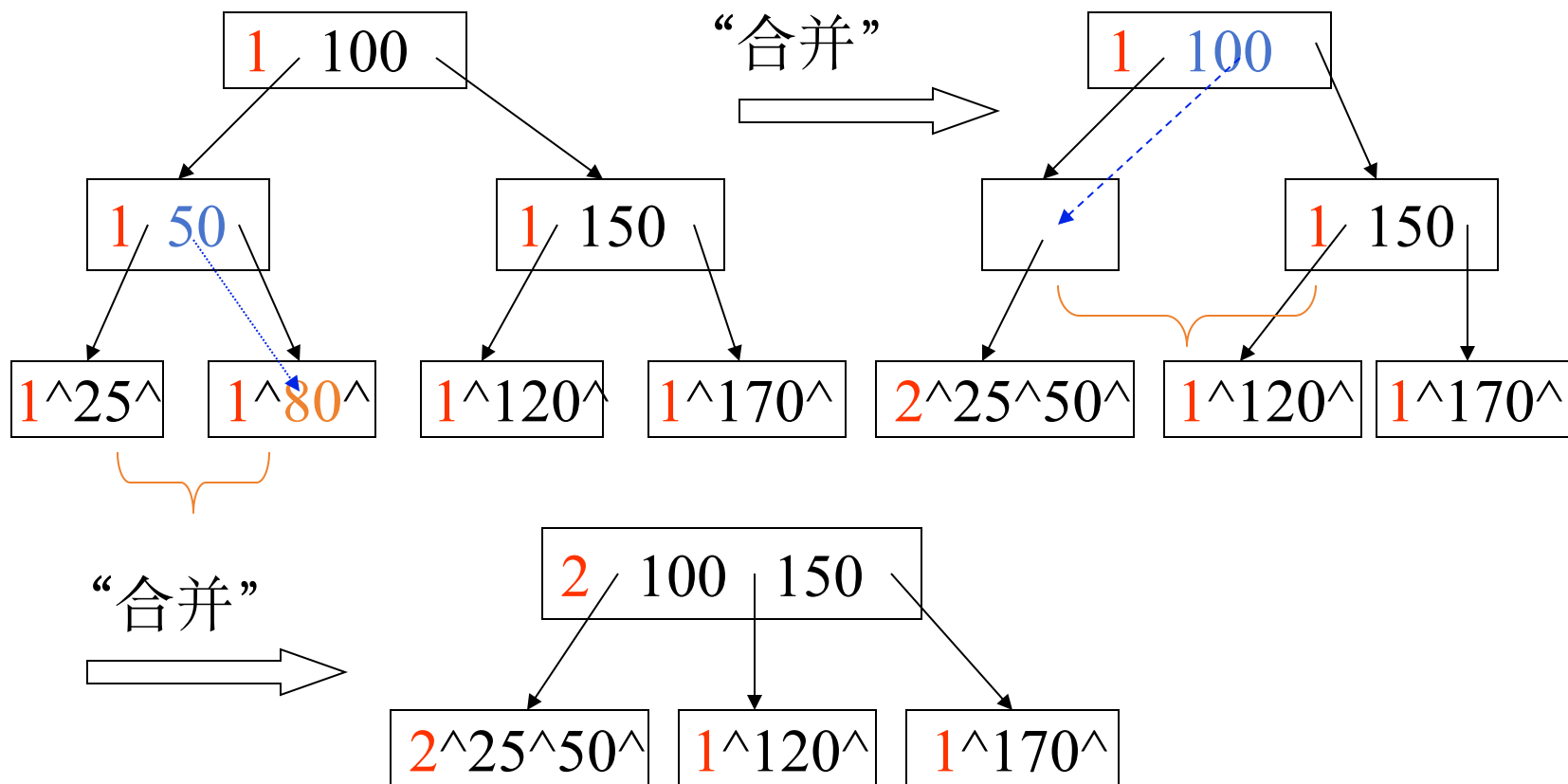
- 其所属结点的关键字个数= $\lceil m/2 \rceil - 1$,
其左/右兄弟结点关键字个数= $\lceil m/2 \rceil - 1$

自己不够, 兄弟也不够



“连接”时出现的较为复杂情况的处理:

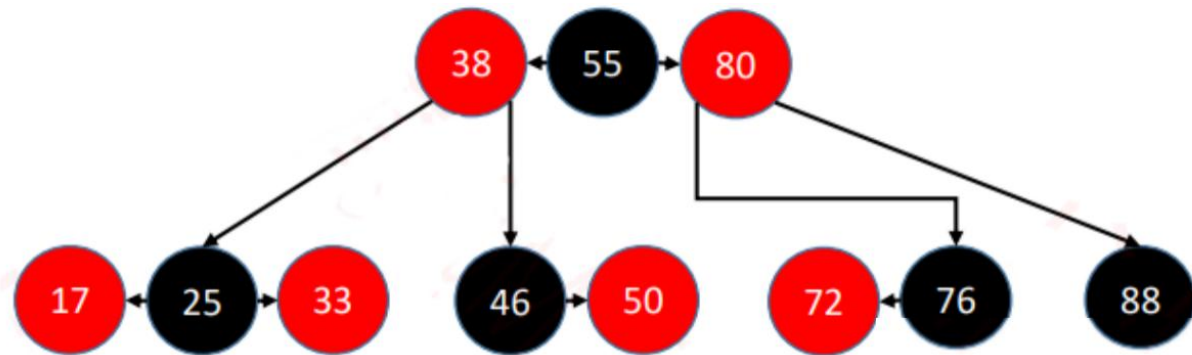
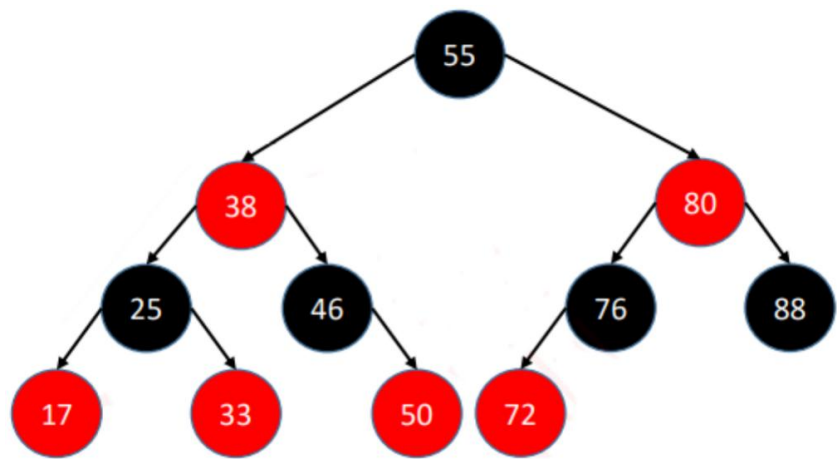
m=4, 删除K=80



5. B-树的高度及性能分析

- **定理** 对于任一棵具有 $n(n>0)$ 个关键字的 m ($m>2$)阶B-树, 其树高 h 至多为 $\log_t((n+1)/2)+1$ 。其中 $t=\lceil m/2 \rceil$
- 决定B-树上操作的时间开销的两个主要因素
 - 外存的访问次数 $O(h)$ —— $O(\log_t n)$
 - CPU的在结点上的比较定位时间 $O(hm)$ 或 $O(h \cdot \log_2 m)$
前者起主导作用
- 作为外存上的动态查找表, m 越大, 则B-树的高度越小, 时间性能越好; 仅在内存中使用的B-树必须取较小的 m , 通常取最小值 $m=3$

4阶B-树与红黑树



- 左图是红黑树，省略了值为空的黑色叶子结点，
- 将左图所有的红色结点上移到和他们的父结点同一高度上，转化成下图所示的四阶B-树（2-3-4树）
- 红黑树和4阶B树（2-3-4树）具有等价性，在二叉树的属性中加入一个颜色属性来表示2-3-4树中不同的结点。

通过染色、旋转调平

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

9.2.4 B+树

B+树是一种**B-树的变形树**。支持按区间查询，是MySQL等数据库使用的主要索引类型之一。

1. m阶B+ 树与 B-树的不同之处

- 1)有 n 棵子树的结点中有 n 个关键字;
- 2) 非叶结点可以看成是索引部分 **索引集**

A_i : 第 i 个子结点的指针

K_i : 第 i 个子结点的最大 (或最小) 关键字

- 3)所有叶子结点中包含了全部关键字的信息及指向这些关键字记录的指针，且叶子结点以关键字大小自小至大顺序链接；**数据集**

<https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>

[结点结构]

非叶结点 ($A_1, K_1, \dots, A_i, K_i, \dots, A_n, K_n$) 索引集

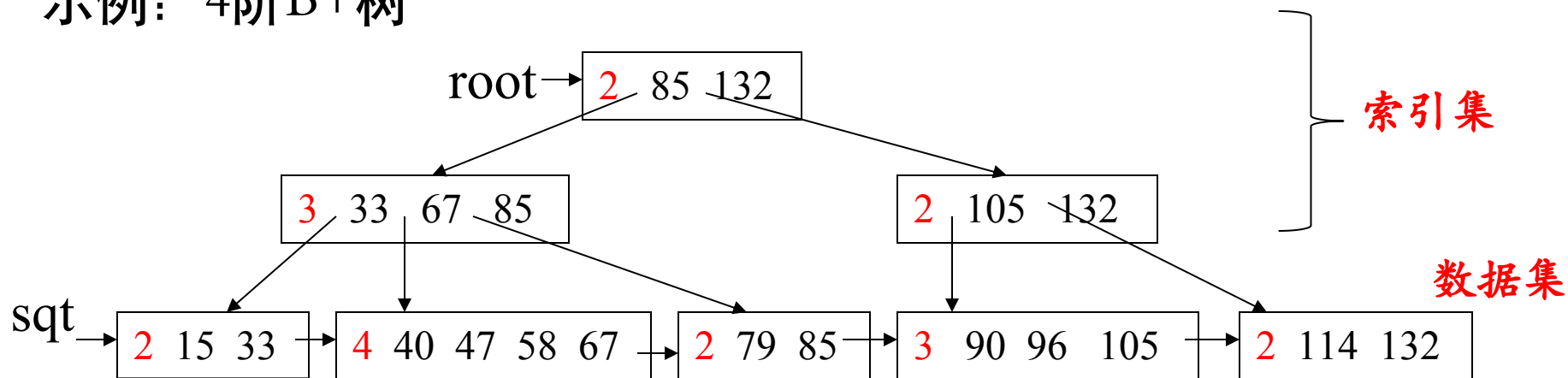
A_i : 第 i 个子结点的指针

K_i : 第 i 个子结点的最大 (或最小) 关键字

叶结点

全部关键字及指向关键字记录的指针 数据集

示例：4阶B+树



2. B+树上的基本运算

1)查找

方式1: 从根结点开始, 利用索引集结构, 向下查找直到叶子结点

方式2: 从最小关键字开始, 沿叶结点数据集的链结构顺序查找

2)插入

仅在叶子结点上进行, 关键字个数大于 m 则分裂

3)删除

也仅在叶子结点上进行, 关键字个数小于 $\lceil m/2 \rceil$ 时, 需进行合并

9.2.5 键树(数字查找树/字符树)

1. 概念

<https://www.cs.usfca.edu/~galles/visualization/Trie.html>

将关键字**分解**为字符的多叉树，多叉树中的**每个结点只代表关键字中的一个字符**，**叶结点**用于表示字符串的结束符(如 ‘\$’)，并**含有指向该关键字记录的指针**。从根到叶子的**路径上所有的结点对应的字符连接起来就代表一个关键字**。约定

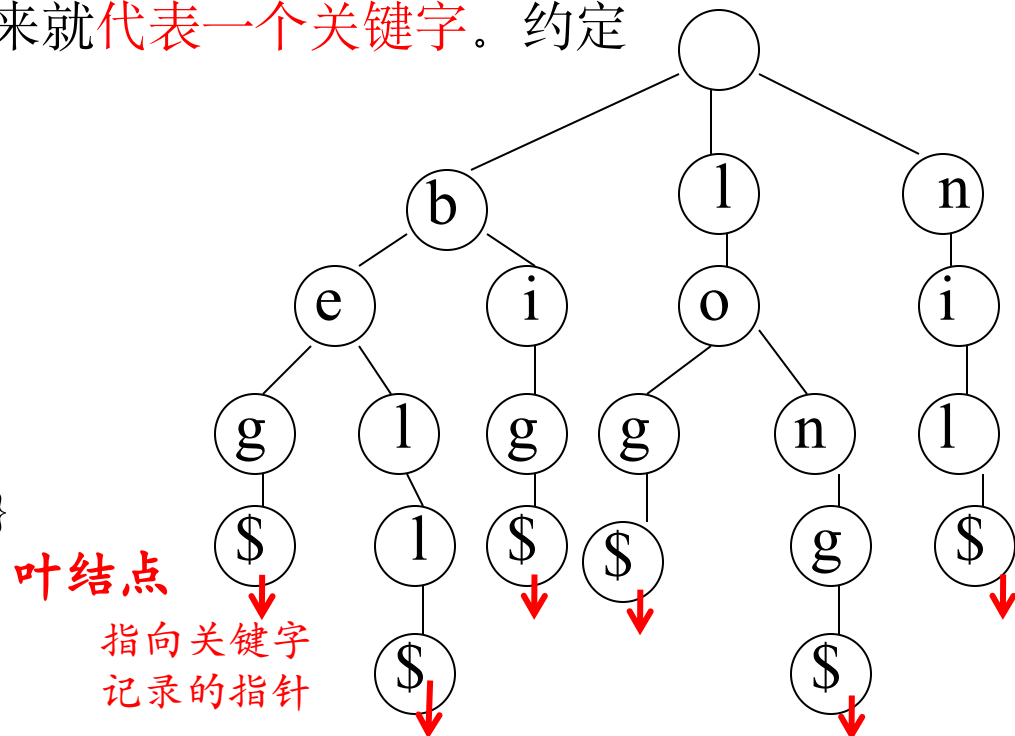
键树是**有序树**

结束符小于任何字符

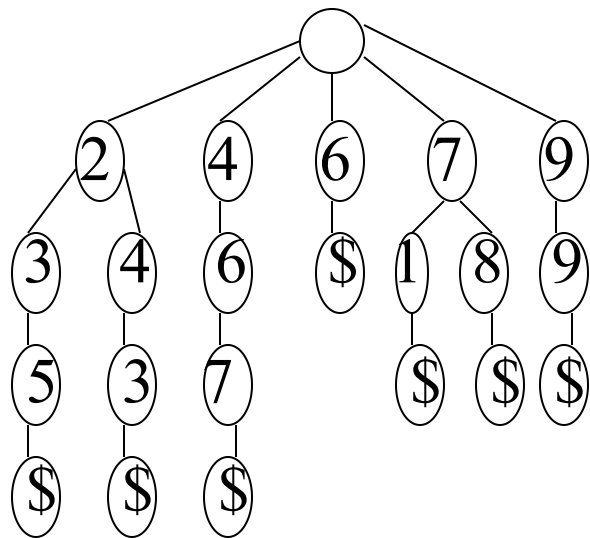
[例1]

关键字集合

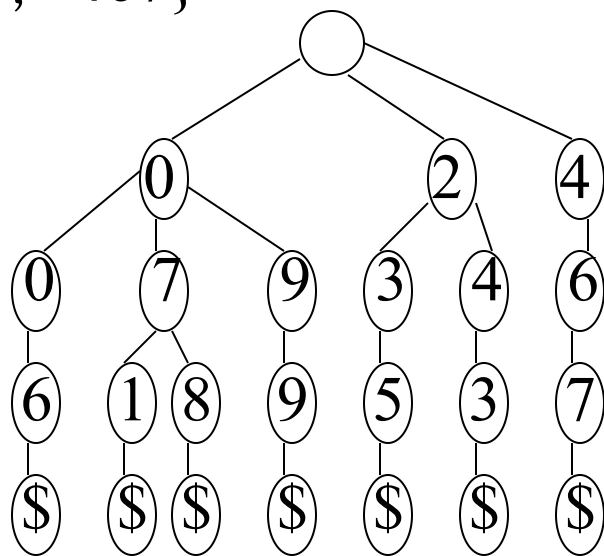
{beg, bell, big, log, long, nil}



[例2] {6, 71, 78, 99, 235, 243, 467}



按**纯字符串**考虑

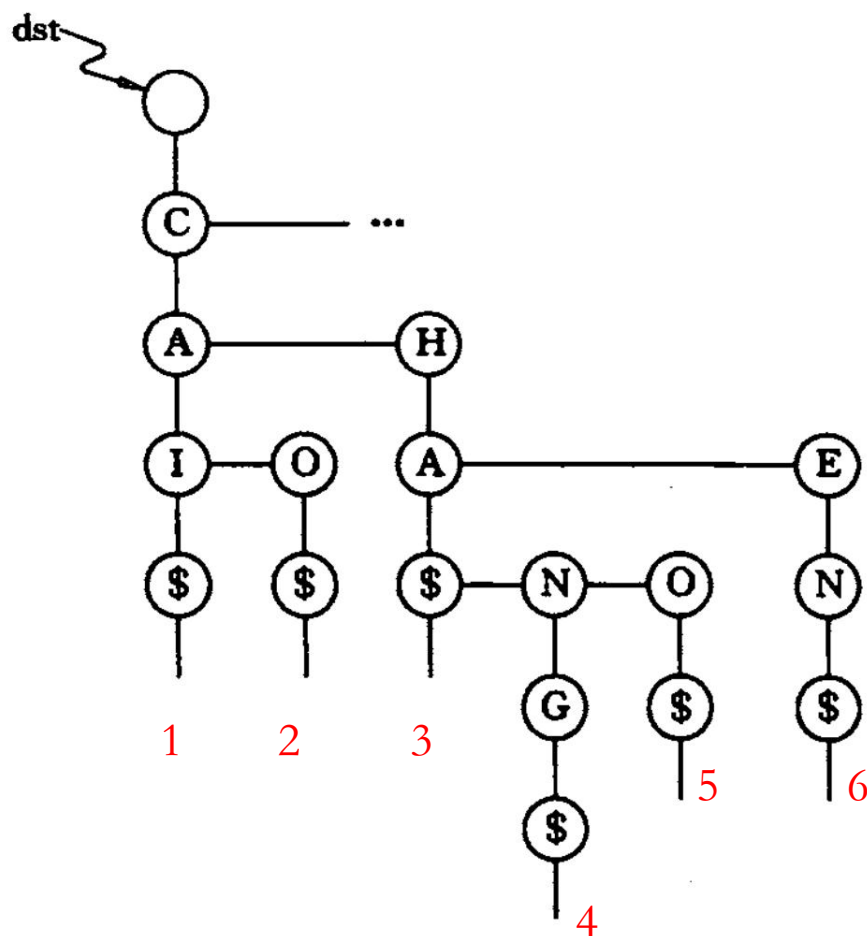


按**整型数**考虑

2. 键树的存储结构及其操作

1) 双链树

——以树的孩子兄弟链表表示



symbol	ptr	next
--------	-----	------

字符

兄弟指针

结点为leaf型: 指向记录的指针

结点为nonleaf型: 指向孩子指针

1: CAI

2: CAO

3: CHA

4: CHANG

5: CHAO

6: CHEN

[查找]

设给定值为 $K[1..m]$

成功查找的情况 $K = \text{'CHEN\$'}$

从 $p = \text{dst} \rightarrow \text{ptr}$ 出发

沿next域顺序查找，直到某个结点的symbol值等于 $K[1]$;

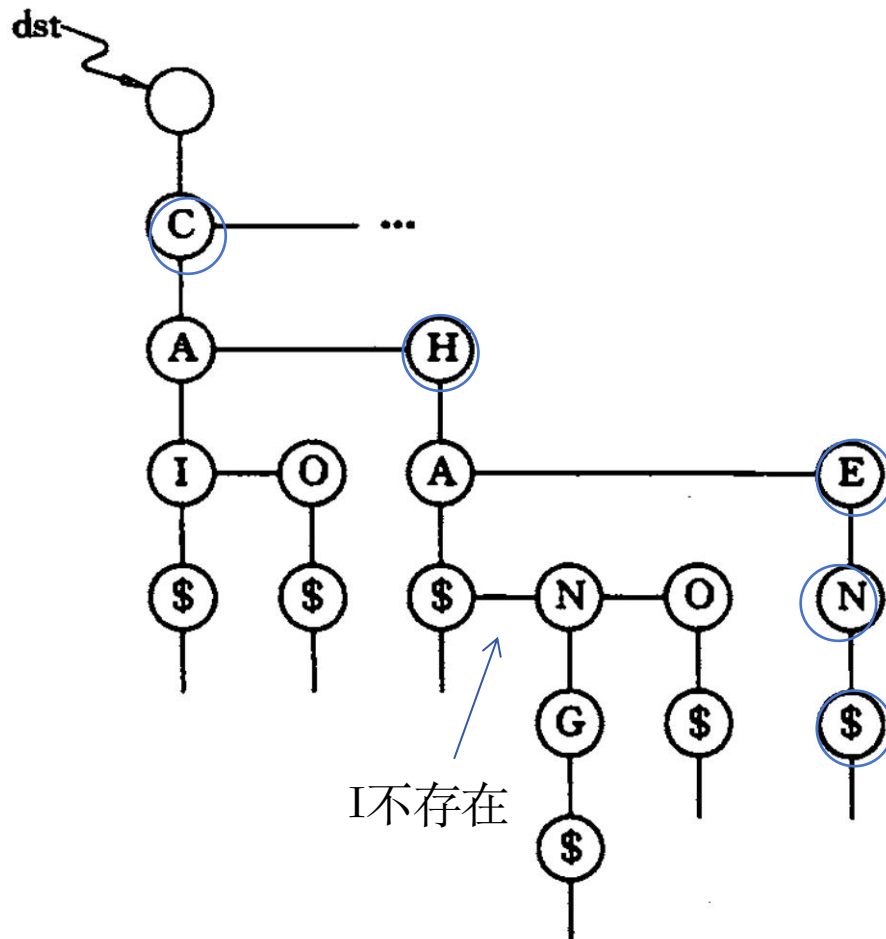
再沿该结点的ptr域进入下一层结点，查找 $K[2]$;

直至 $K[M]$ 比较在叶结点处终止。

$$\text{ASL} = h(1+d)/2$$

d —键树中结点的最大度

不成功查找的情况 $K = \text{'CHAI'}$



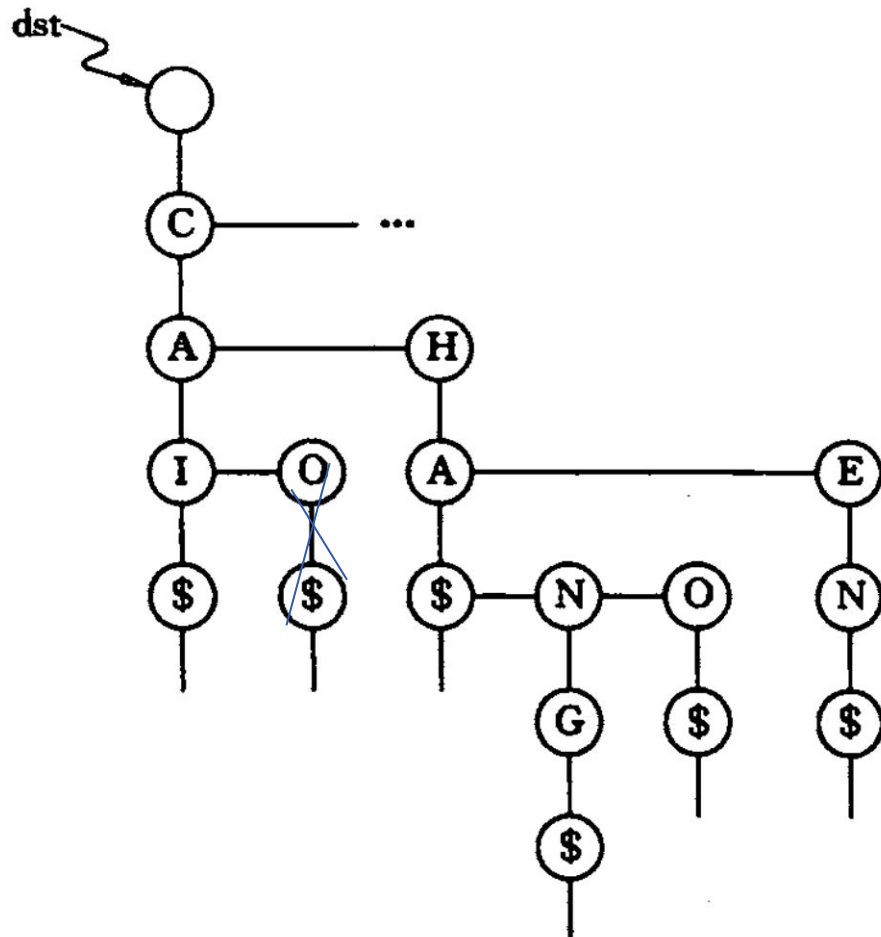
[插入] 查找不成功时，插入

例：插入CHAI

[删除] 确认查找成功后，删除

例：删除CAO

注意：结点只有一个孩子或是叶结点
时才可删除。



2) Trie树(来源于retrieval(检索))——以多重链表表示

若以多重链表表示键树，则树的每个结点含有d个指针域，此时的键树称为Trie树。

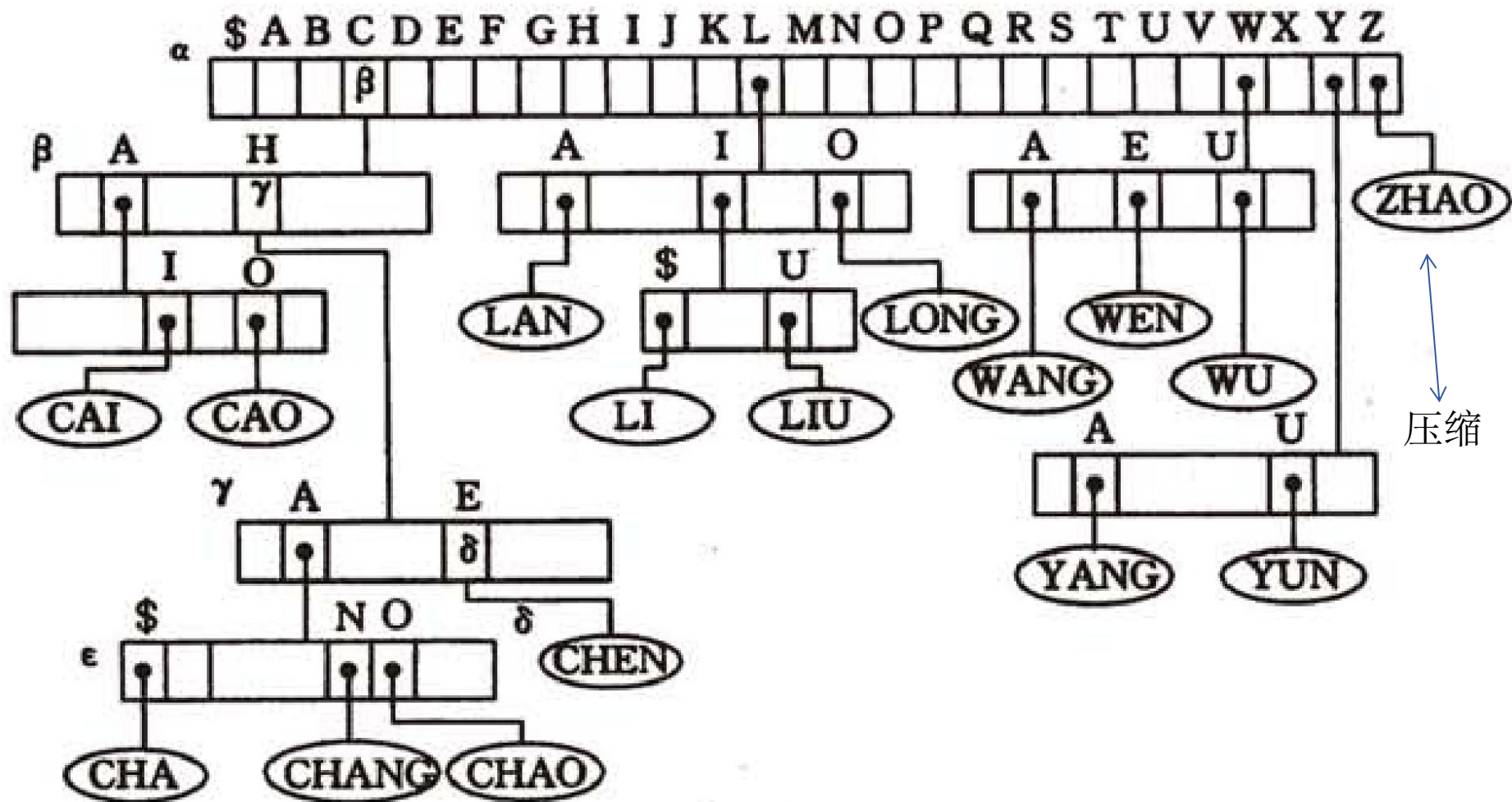
分支结点： num—有用指针的数量

ptr[0..m]—指针数组

叶子结点： key—关键字

recptr—指向对应记录的指针

若从键树中某个结点开始到叶子结点的路径上的每个结点中都只有一个孩子，则将该路径上的所有结点压缩成一个“叶子结点”，且叶子结点中存储有指向记录的指针。



压缩

9.3 哈希表

9.3.1 概述

9.3.2 哈希函数的基本构造方法

9.3.3 处理冲突的常用方法

9.3.4 哈希表的建立、查找及其ASL分析

9.3.1 概述

之前各种查找表的结构特点：

- 记录在表中的位置和它的关键字之间不存在一个确定的关系
- 查找的过程为给定值依次和集合中各个关键字进行比较，查找的**效率取决于**和给定值进行**比较**的关键字个数。
- 不同的查找方法，其差别仅在于：关键字和给定值进行比较的顺序不同。

[哈希表的基本思想]

在记录的**存储地址**和它的**关键字**之间**建立**一个确定的**对应关系**；这样，理想状态不经过比较，一次存取就能得到所查元素。

[术语]

哈希表 一个有限的连续的地址空间，用以容纳按哈希地址存储的记录。

哈希函数 记录的存储位置与它的关键字之间存在的一种对应关系。

$$Loc(r_i) = H(key_i)$$

冲突 对于 $key_i \neq key_j, \quad i \neq j,$

出现的 $H(key_i) = H(key_j)$ 的现象。

同义词 在同一地址出现冲突的各关键字。

哈希(散列)地址 根据设定的哈希函数 $H(\text{key})$ 和处理冲突的方法确定的记录的存储位置。

装填因子 表中填入的记录数 n 和哈希表表长 m 之比。

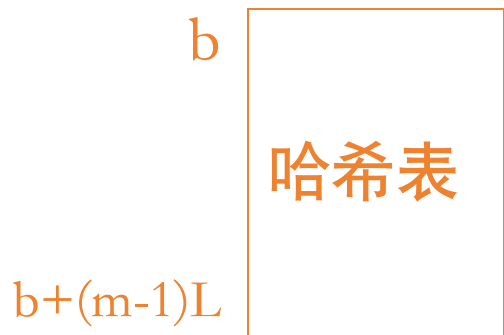
$$\alpha = n/m$$

[设计哈希表的过程]

1)明确哈希表的地址空间范围。即确定哈希函数的值域。

2)选择合理的**哈希函数**。该函数要保证所有可能的记录的哈希地址均在指定的值域内，并使冲突的可能性尽量小。

3)设定**处理冲突**的方法。



9.3.2 哈希函数的基本构造方法

构造原则： 算法简单，运算量小；
均匀分布，减少冲突。

若是非数字关键字，则需先对其进行数字化处理。

[直接定址法] $H(key)=a * key + b$ a 、 b 为常数

特点： 计算简单，冲突最少。

此法仅适合于： 地址集合的大小 = = 关键字集合的大小

[数字分析法] 取关键字的若干数位作为哈希地址。

假设关键字集合中的每个关键字都是由 s 位数字组成 $(u_1, u_2, \dots, u_s)$ ，分析关键字集中的全体，并从中提取分布均匀的若干位或它们的组合作为地址。

此方法仅适合于： 能预先估计出全体关键字的每一位上各种数字出现的频度。

[平方取中法] 取关键字平方后的中间几位作为哈希地址

求“关键字的平方值”的目的是“扩大差别”，同时平方值的中间各位又能受到整个关键字中各位的影响。

此方法适合于：关键字中每一位都有某些数字重复出现频度很高的现象。

[折叠法]将关键字分割成位数相同的几部分(最后一部分的位数可以不同)，然后取这几个部分的**叠加和**(舍去进位)作为哈希地址。

此方法适合于：关键字的数字位数特别多。

例：key = 110108380428895

895		895
428	移位	824
380	叠加	380
108		801
+) 110		+) 110
1921		3010

[除留余数法] $H(key) = key \text{ MOD } p \quad (p \leq m)$

m: 哈希表的表长;

p: 最好为素数, 更有利于“散列”

为什么要对 p 加限制?

给定一组关键字为: 12, 39, 18, 24, 33, 21,

若取 $p=9$, 则对应的哈希函数值将为:

3, 3, 0, 6, 6, 3

可见, 若 p 中含质因子 3, 则所有含质因子 3 的关键字均映射到“3 的倍数”的地址上, 从而增加了“冲突”的可能。

[随机数法] $H(key) = \text{random}(key)$

此方法适合于: 关键字长度不等时的情况。

9.3.3 处理冲突的常用方法

为产生冲突的地址寻找下一个哈希地址

1. 开放定址法 (空缺编址法)

$$H_i = (H(\text{key}) + d_i) \text{ MOD } m$$

$$i=1, 2, \dots, k \quad (k \leq m-1)$$

m : 哈希表的表长; d_i : 增量序列



1) 线性探测再散列 $d_i = 1, 2, \dots, m-1$

缺陷: 有聚集(堆积)现象—非同义词地址冲突。

2) 二次探测再散列 $d_i = 1^2, -1^2, 2^2, -2^2, 3^2, \dots, \pm k^2$

$$k \leq m/2$$

缺陷: 不易探查整个散列空间。

3) 伪随机探测再散列 $d_i = \text{伪随机数序列}$

4) 双重散列函数探查法 $d_i = i * h_1(\text{key})$

$$0 < i \leq m - 1$$

要求: $h_1(\text{key})$ 的值与 m 互素。

m 为素数时, $h_1(\text{key})$ 可取1到 $m-1$ 之间的任何数,

建议 $h_1(\text{key}) = \text{key} \bmod (m-2) + 1$

m 为2的方幂时, $h_1(\text{key})$ 可取1到 $m-1$ 之间的任何奇数

开放定址法不宜执行删除操作



2. 再哈希法

$$H_i = RH_i(\text{key}) \quad i=1, 2, \dots, k$$

RH_i 为不同的散列函数

例如: 关键字集合 { 19, 01, 23, 14, 55, 68, 11, 82, 36 }

设定哈希函数 $H(key) = key \text{ MOD } 11$ (表长=11)

若采用线性探测再散列处理冲突

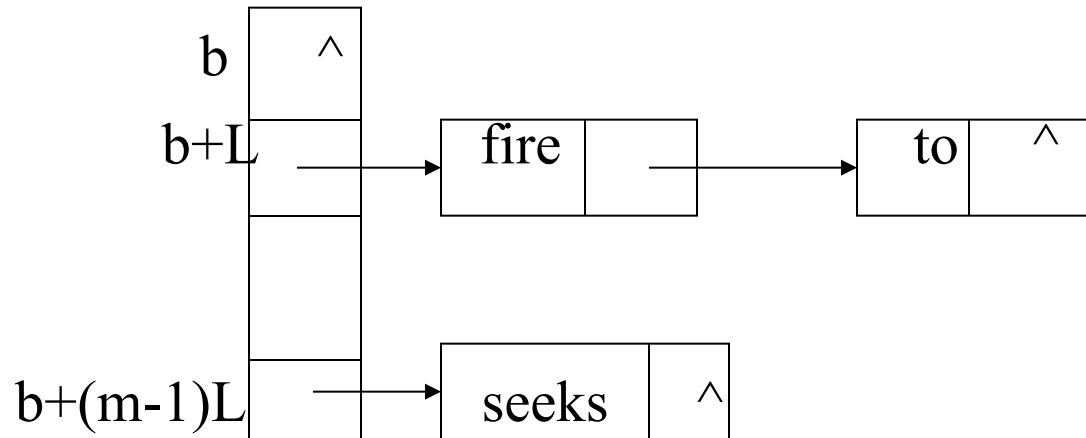
0	1	2	3	4	5	6	7	8	9	10
55	01	23	14	68	11	82	36	19		
1	1	2	1	3	6	2	5	1		

若采用二次探测再散列处理冲突

0	1	2	3	4	5	6	7	8	9	10
55	01	23	14	36	82	68		19		11

3. 链地址法

为每个哈希地址建立一个单链表，存储所有具有同义词的记录。



冲突处理简单，无堆积现象，平均查找长度较短；

较适合于事先无法确定表长的情况；

可取 $\alpha \geq 1$ ，当结点信息规模较大时，节省空间

删除结点的操作易于实现

4. 建立一个公共溢出区

<https://www.cs.usfca.edu/~galles/visualization/ClosedHashBucket.html>

基本表

溢出表

9.3.4 哈希表的建立、查找及其ASL分析

[例] 有一组关键字序列(38、59、125、168、216、719、455、20)，用函数 $H(\text{key}) = \text{key} \text{ MOD } 10$ 将其按顺序散列到哈希表 HT(0:9) 中，分别用三种方法解决冲突：线性再探测法、链地址法、公共溢出区法，画出这三种方法建立的散列表，并分别求在等概率情况下查找成功和不成功的平均查找长度。

$$ASL = \sum_{i=1}^n p_i C_i$$

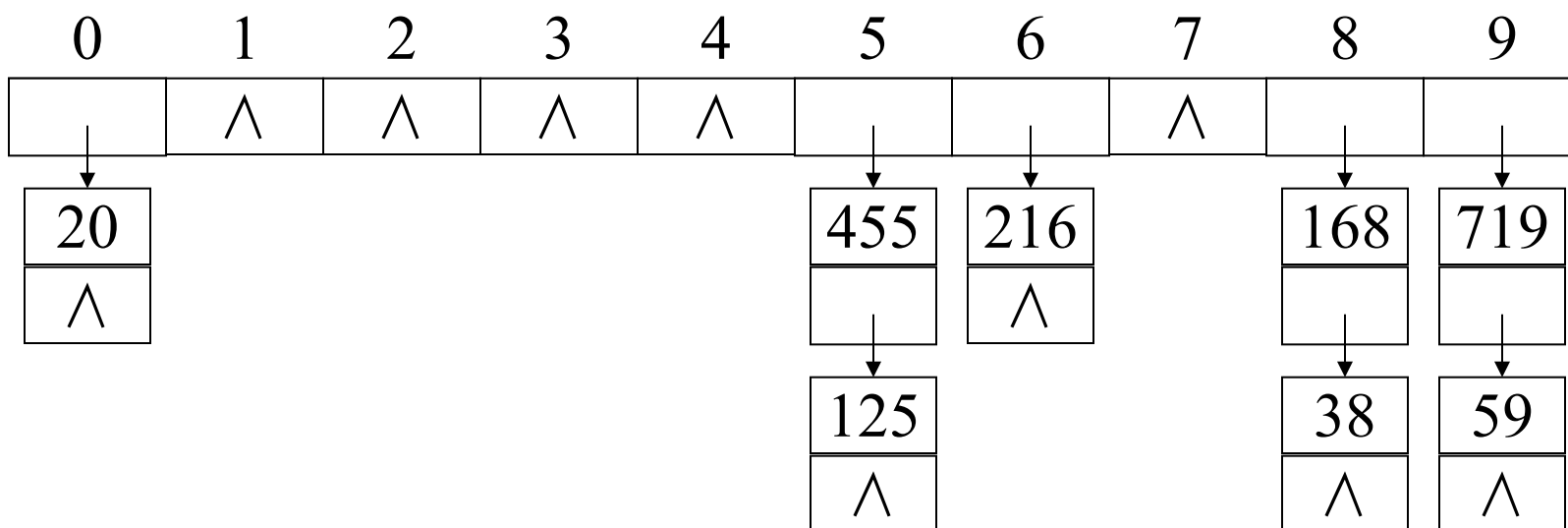
1) 线性再探测法

0	1	2	3	4	5	6	7	8	9
168	719	20			125	216	455	38	59
719	20				455	455		168	168
3	3	3			1	1	3	1	1

查找成功： $ASL = (3+3+3+1+1+3+1+1)/8$

查找失败： $ASL = (4+3+2+1+1+9+8+7+6+5)/10$

2)链地址法



拉链法处理冲突时的哈希表（向链表中插入元素均在表头进行）

查找成功: $ASL = (1 \times 5 + 2 \times 3) / 8 = 11 / 8$

查找失败: $ASL = (1 + 2 + 1 + 2 + 2) / 10$

3. 建立一个公共溢出区

设哈希函数产生的哈希地址集为 $[0, m-1]$ ，则分配两个表：

基本表 `DateType base_t[m]`: 每个单元只能存放一个元素；

溢出表 `DateType over_t[k]`: 只要关键码对应的散列地址在基本表上产生冲突，则所有这样的元素一律存入该表。

查找时，对给定关键码 k_x 通过散列函数计算出散列地址 i ，先与基本表的`base_t[i]`单元比较，若相等，查找成功，否则，到溢出表进行查找。

3)公共溢出区法

0	1	2	3	4	5	6	7	8	9
20					125	216		38	59
					455		168 719		

	0	1	2	3	4	5
公共溢出区:	168	719	455			
	719	455				

查找成功: $ASL = (1 \times 5 + 2 + 3 + 4) / 8 = 14 / 8$

查找失败: $ASL = (5 \times 5 + 5 \times 1) / 10 = 30 / 10$

散列表上的查找

散列表的查找过程基本上和造表过程相同。

一些关键码可通过散列函数转换的地址直接找到，另一些关键码在散列函数得到的地址上产生了冲突，需要按处理冲突的方法进行查找。

查找效率取决于产生冲突的多少，产生的冲突少，查找效率就高，产生的冲突多，查找效率就低。影响产生冲突有以下3个因素：

- 散列函数是否均匀
- 处理冲突的方式
- 散列表的装填因子

第三个因素：哈希表的装填因子。装填因子是散列表装满程度的标志因子，一般选择在0.65-0.85范围。由于表长是定值，与“填入表中的元素个数”成正比，所以，越大，填入表中的元素较多，产生冲突的可能性就越大；越小，填入表中的元素较少，产生冲突的可能性就越小。

实际上，散列表的平均查找长度是装填因子的函数，只是不同处理冲突的方法有不同的函数。下页表给出几种不同处理冲突方法的平均查找长度和的关系，可供参考。

散列表上的删除

当在散列表上删除一个元素时，首先是查找，查找成功情况下才能做删除。

- 对于拉链法解决冲突构造的散列表，其删除等价于单链表上的删除；
- 对于开放地址法解决冲突构造的散列表，不能简单的将删除元素所在单元置为空，这样会断掉原来的探测地址序列，查找后面的元素将受到影响，删除这个元素可以将这各单元置为有别于空单元表示的特殊值，在查找时遇到这个特殊值，需要继续探测序列。

[平均查找长度与哈希表的装填因子的关系]

$$\alpha = \frac{\text{表中填入的记录数}}{\text{哈希表的长度}}$$

在一般情况下， $\alpha \downarrow$ ，冲突的可能性 $\downarrow$ ，ASL $\downarrow$ ，但空间的浪费 $\uparrow$ 。

	成功ASL	失败ASL
线性探测再散列	$\frac{1}{2}(1 + \frac{1}{1 - \alpha})$	$\frac{1}{2}(1 + \frac{1}{(1 - \alpha)^2})$
伪随机探测再散列	$-\frac{1}{\alpha} \ln(1 - \alpha)$	$\frac{1}{1 - \alpha}$
链地址法	$1 + \frac{\alpha}{2}$	$\alpha + e^{-\alpha}$